

aXet.flows: A platform to automate IT services quickly and easily

A flow-based system that interacts with any system and application, communicates with users, controls devices using RPA techniques, and applies AI for decision-making.

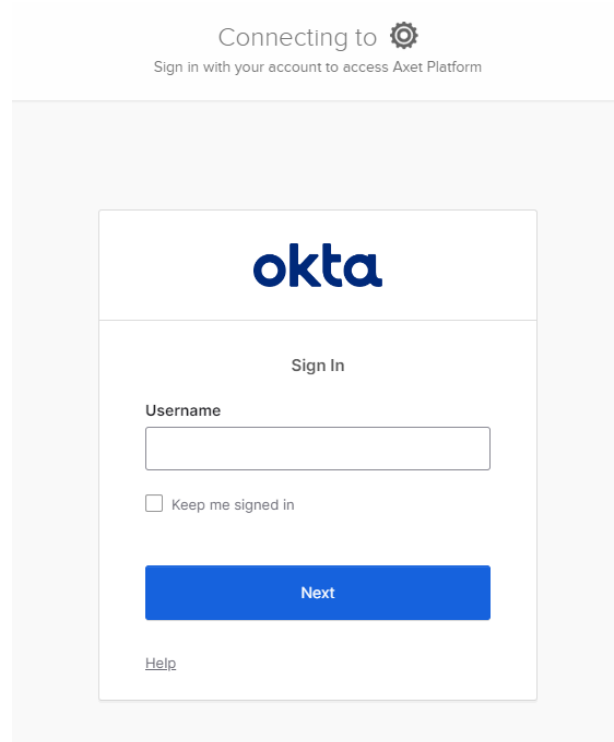
It is part of the aXet platform.

aXet.flows - First steps & Installation

Access aXet.flows

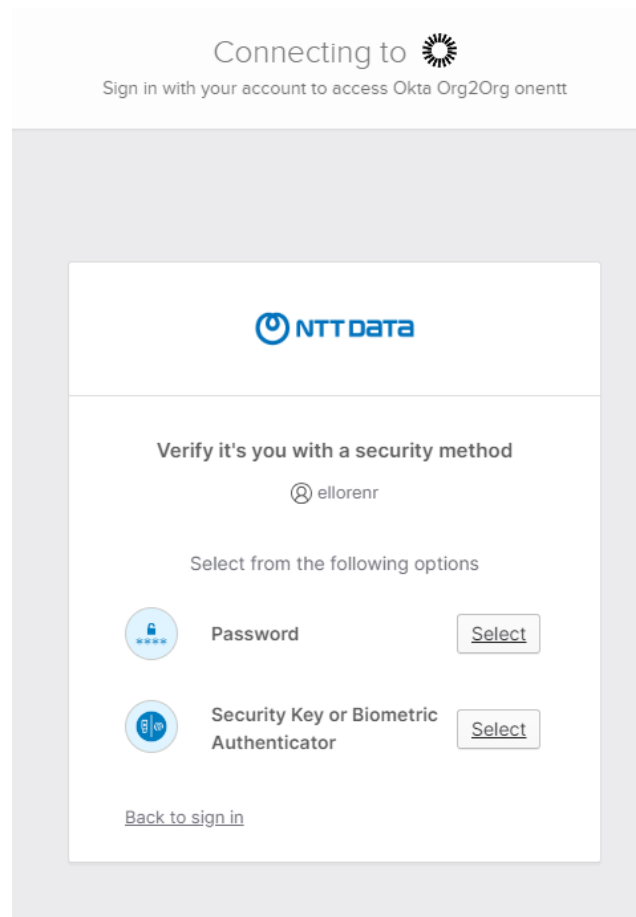
First step is to access the [web interface](#).

1. Log in to Okta platform. You can mark the *'Keep me signed in'* check.



The screenshot shows a web interface for connecting to the Axet Platform. At the top, it says "Connecting to" with a gear icon, followed by "Sign in with your account to access Axet Platform". Below this is a white box containing the Okta logo, the text "Sign In", a "Username" label, a text input field, a checkbox labeled "Keep me signed in", a blue "Next" button, and a "Help" link at the bottom left.

2. Select the option to verify your credentials.



3. Once logged into Okta, you can now access **aXet.flows**.
4. If it is your first time accessing to aXet, otherwise, you can skip this step:
 - a) Select the project you are going to work in.
 - b) If you do not find your project in the list, you can request the creation of a new one.
 - c) Once you have a project, Click on 'Save' button to start **aXet.flows**.

Install aXet.flows Installer

The second step is to install the **aXet.flows** installer.

Minimum System Requirements

You must have a PC with at least the following specifications:

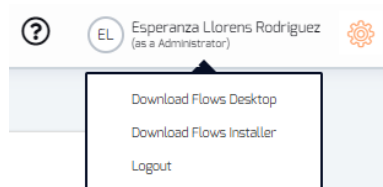
- The **hardware** of the equipment must meet the following characteristics:
 - 64-bit processor with Second Level Address Translation (SLAT).

- Hardware virtualization support must be enabled in the BIOS settings. For more information, click [here](#).
- At least 8 GB of RAM.
- Between 8-10 GB of free disk space to carry out the installation of the various tools mentioned below.
- **Operating System:**
 - Windows 10 64-bit: Pro, Enterprise, or Education (Build 15063 or higher).
- **Software:**
 - Google Chrome internet browser, updated to the latest stable version available.

Installation

Below, the steps you must follow to correctly carry out the installation of the tool are detailed so that you can design your own flows.

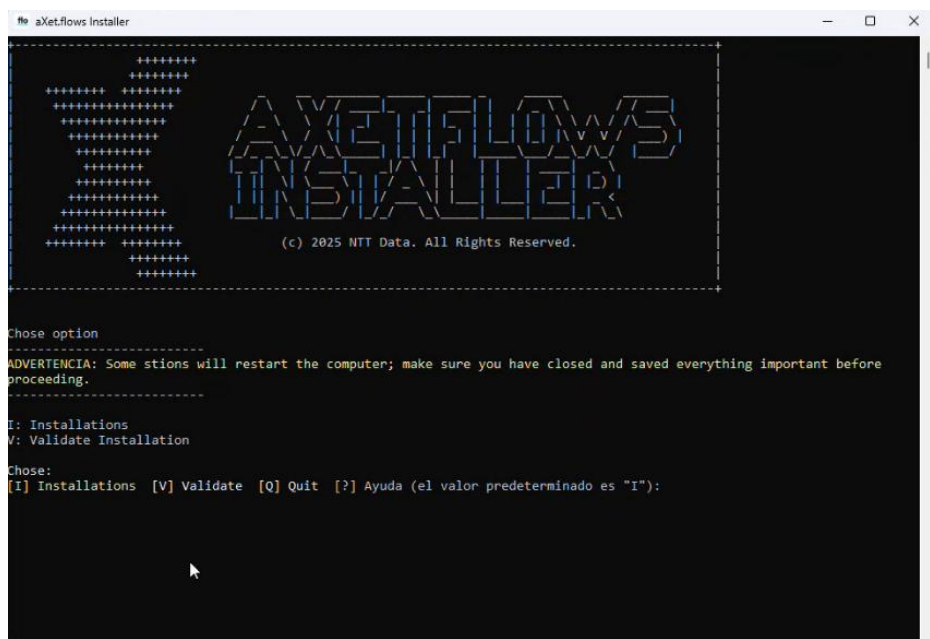
You can obtain the necessary installer by clicking on your (previously registered) user and selecting the '*Download Flows Installer*' menu item.



Remember that you must have the administrator password to carry out the installation.

These are the steps to install **aXet.flows**.

1. Run the **aXet-flows_installer.exe** file to start the installation process.
2. A PowerShell instance will open where you need to enter your username (short name) and password.
4. After validating your credentials, select the option to start the installation process.



5. Choose **I** for starting installation.

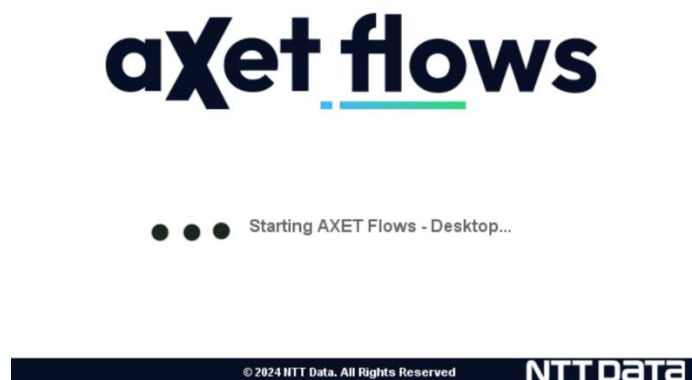
6. This process will install also the required tools:

Component Map

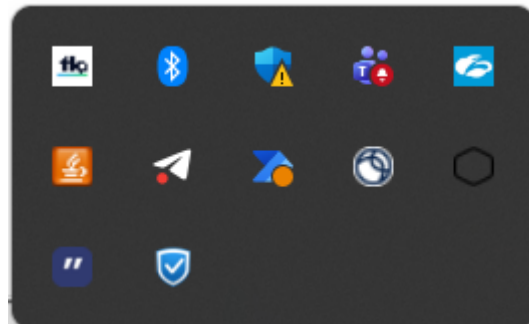
Component	Validation Function	Installation Function	Resource(s) Used	Notes
wslenv	ValidateWslEnv	InstallWslEnv	(none)	Sets system variable WSL_UTF8=1 for UTF-8 support in WSL.
wsl	ValidateWslFeature	InstallWslAdminFeatures	(managed in aXet.flows desktop)	Enables required Windows features for WSL.
node	ValidateNode	InstallNode	node.msi	Installs Node.js 20.
zscaler	ValidateZscalerForNodejs	InstallZscalerNodeFix	Zscaler_Root_CA.der.b64, Zscaler_Root_CA.pem	Adds Zscaler certs and env variable for Node.js.

Run aXet.flows Desktop

The third step, before starting to design flows, is to run **aXet.flows-Desktop.exe**. You can download the executable from the menu that opens when you click on your username. It is the same menu where you downloaded the **aXet.flows installer**.

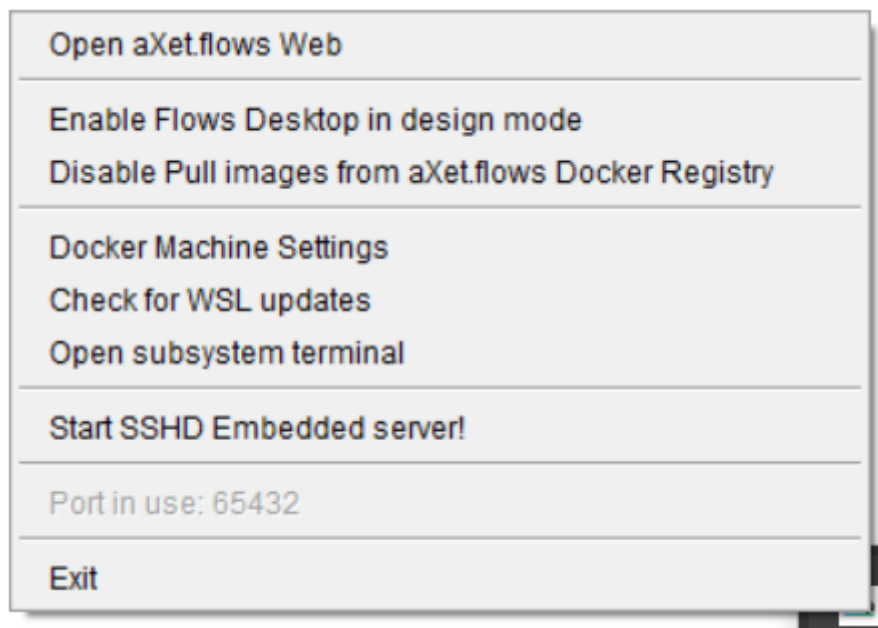


You can find the **aXet.flows** running icon in the systray.



You already have all the tools installed and you are ready to start using **aXet.flows**.

Launch it from the context menu, select **Open aXet.flows Web**.



aXet.flows – Interface

Let's introduce the graphical interface of **aXet.flows**.

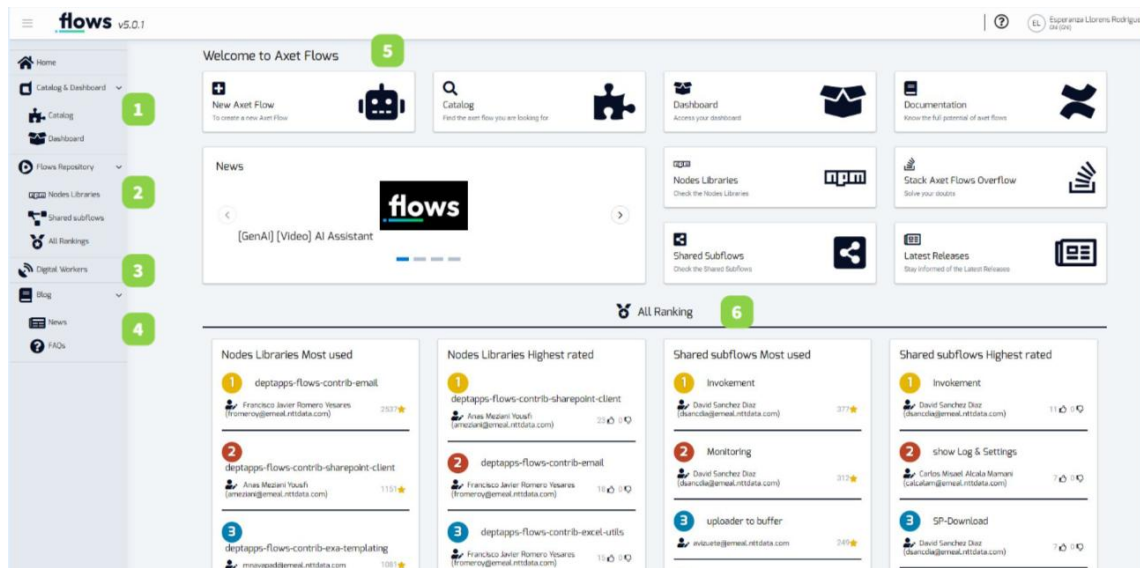
Select if you want to know the web interface that allows you to manage development projects or the web interface that allows you to design it.

Home Page

Let's introduce the graphical interface of **aXet.flows**.

Here you can see the home page of **.flows**.

You can click on the numbers to get the description of the main elements or you can read it below the image.



These are the points highlighted in the image:

1. Catalog & Dashboard.

By clicking on this section, you can access the **Catalog** of development projects that you own and/or have access to, and the **Dashboard** that shows in real-time the instances that are running.

2. Flows Repository.

In this section, you have access to:

- a. **Nodes Libraries.** Here you can find the nodes created by **aXet.flows** users that extend the possibilities it offer.

- b. **Shared Subflows.** A subflow is a collection of nodes that are collapsed into a single node. Here you can find the subflows developed by **aXet.flows** users.
- c. **All Rankings.** Here you can find the most used and highly rated node libraries and shared subflows by **aXet.flows** users.

3. Digital Workers.

In this section, you can monitor the status of your development projects that use 'Worker' nodes for unattended execution.

4. Blog.

In this section, you can find information about **.flows**, such as a blog and FAQs.

5. Shortcuts.

In this section, you can find shortcuts to the most used tools, as well as information related to **aXet.flows**.

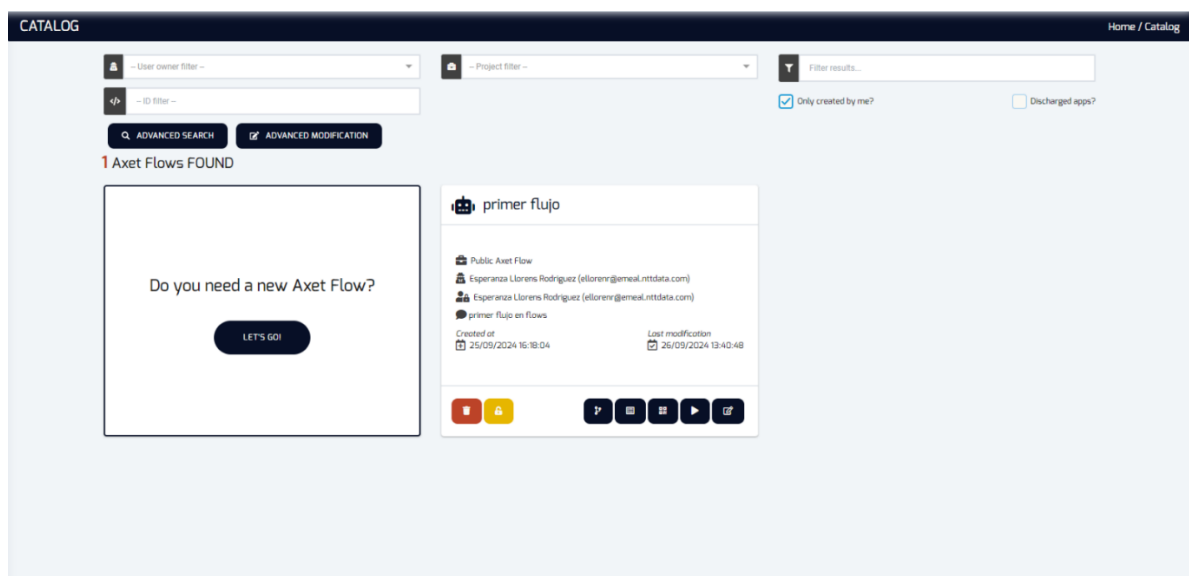
6. All Ranking.

In this section, you can find different rankings related to nodes and subflows most used and highly rated by the users of **aXet.flows**.

Catalog & Dashboard

Catalog

Let's introduce **Catalog** section.



You can visualize all the information of the different development projects that, either you own them, or you have access to execute/evolve them. For each of the development projects, you have the following information on the screen:

- **Name** at the header of the card.
- **Project** to which the development project has been assigned. You can find:
 - Public development projects, without an assigned project.
 - Development projects that have a project assigned. These will only be accessible by those users who are members of the project.
- **User** who has the development project locked. Until that user unlocks the development project, it cannot be evolved by another user who has access to it.
- **Description** of the development project.
- **Creation date** of the development project.
- **Date of the last modification.** The following actions on the development project are considered a modification:
 - Modification of the metadata that identifies the development project.
 - Creation of new versions of the development project.
 - Addition/modification of the meta-information associated with the development project.

For each of the development projects, we will have access to different functionalities through the button panel located at the footer of each card representing an development project. The buttons allow performing the following actions:

- **Delete.** Allows for the logical deletion of the development project in question. This action can only be performed by administrators as well as by the owner of the development project.
- **Lock/Unlock** development project for editing. Allows locking/unlocking the development project to carry out its editing and/or to release it so that it can be edited by another user. The development project should only be locked if there is a desire to evolve its behavior, that is, the development project designed in **aXet.flows** with the aim of evolving/correcting the various functionalities and/or automatisms that this development project adds as added value.

It will not be necessary to lock the development project to modify its meta-information (name, associated project, keywords, ...).

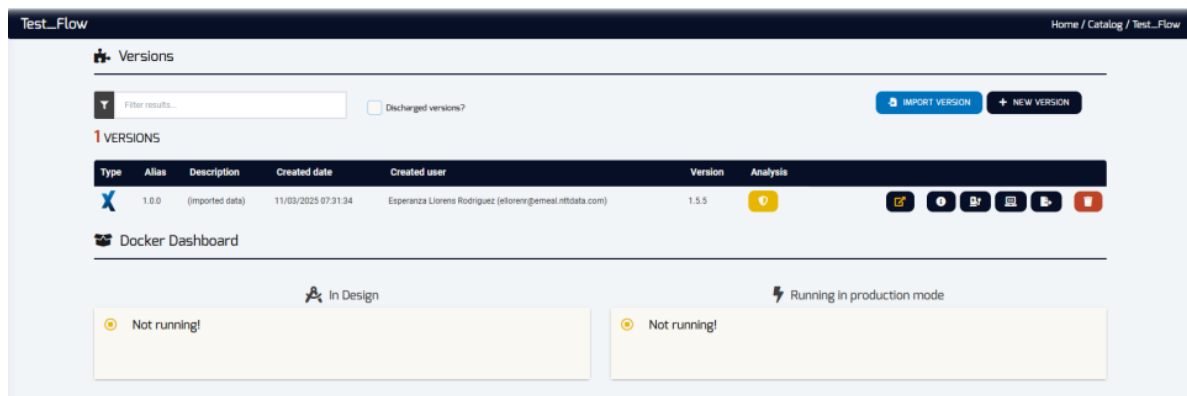
- **Fork.** The concept of fork is exactly the same as found in the lifecycle of a Git repository, on platforms such as GitHub or GitLab. Ultimately, the goal is to make a "clone" of a development project so that it can be evolved by another person, without the original development project being impacted, or simply to create a new development project that starts from a baseline that was created upon the conception of an already existing development project on the platform.

A fork of a development project can only be conducted if it has at least one usable version to execute, that is, if a first usable version of the product has been released.

- **View custom usage log.** Allows viewing the information related to the usage log being performed by a development project.
- **Meta-information aggregator.** Allows access to the dashboard to add/edit the meta-information associated with the development project.
- **Run it!.** Through this button, we will access the development project version management form, which will allow us to:
 - Develop a new version of the development project, starting if it exists from its immediately previous version. For this action, it will be necessary to lock the development project, an action that will be directly proposed by the platform.
 - Execute already developed and consolidated versions of a development project. When running a flow in "production mode", we will not have access to its designer, we will simply be able to use the functionalities it provides, as if it were any other different software that we can use while at the computer.
- **Edit.** Allows modifying the meta-information that conceptually identifies the conception of the development project (name, project, description, keywords) as well as the members, along with the permissions, who have access to the development project.

Dashboard

Let's introduce **Dashboard** section.



This section allows for a global view of all development projects instances that are operational/running on our PC. The platform relies on the **aXet.flows Desktop** component to manage the development projects that are "up", whether in "design mode" or in "production mode":

- **Design mode.** The **aXet.flows** platform will only allow one development project to be up in design mode at any one time.
- **Production mode.** The **aXet.flows** platform allows for different development projects to be up for use. The production mode is designed to use a coded version of the software at the end-user level, without any modifications. At the same time, we can have more than one development project up in production mode on our PC. The only restriction to keep in mind is that we can't run two different versions of the same development project in "production mode".

Here is the information you can get about the development project:

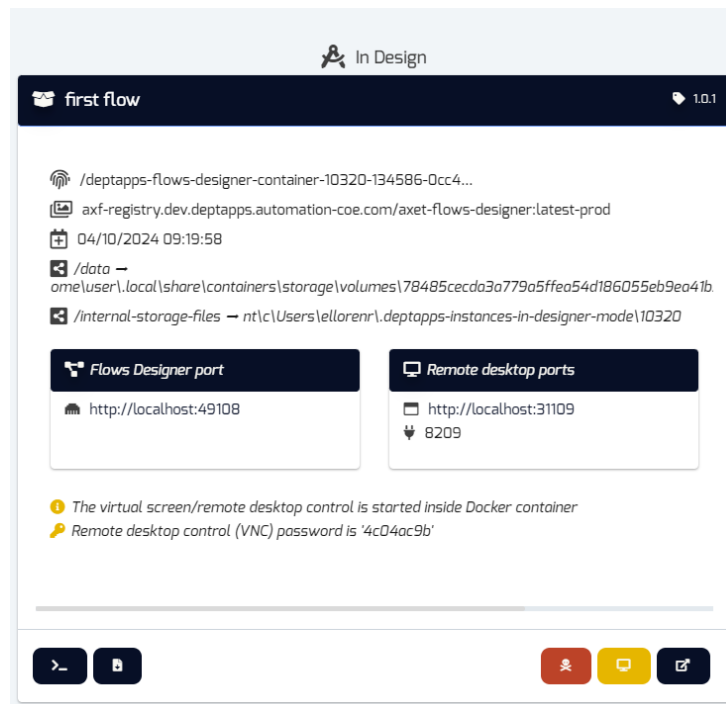
- **Alias** provided to the project.
- Project **Description**.
- **Created date** is the date the project was created.
- **Created user** is the user who created the project.
- **Version** indicates the version numbers of the project, the different versions will be seen in the list.
- **Analysis** informs about the security analysis status of the project. If you click on the icon, the analysis will start. You can find more information about this analysis [here](#). These are the different statuses:
 - Yellow, the project has not been executed.
 - Orange, the analysis is being executed.

- Green, it has successfully completed.
- Red, an issue has been reported that needs to be resolved.

In the last column of the table, you find the icons that allow you to perform the following actions:

- **Edit.** If there are multiple versions, it is only available for the last one.
- Given a version of the development project, it allows viewing relevant meta-information about it, such as which nodes have been used in the construction of the development project.
- **Run this version.** Starts the development project in production mode. It allows running the selected version of the development project developed with **aXet.flows**.
- **Stand-alone deployment.** You have two options: deploy it on a remote server or generate an executable file for Windows 32 and 64 bits, Linux (Debian-based 64 bits), or Mac OS X (64 bits).
- **Export version.** It allows exporting the version of the development project, so it can be imported into the same environment or a different environment. A file with the extension *.deptapp will be downloaded to your PC.
- **Delete** the selected version of the development project.

Ultimately, the **Dashboard** section allows us to view the instances of the different development projects that are up, in one mode or another. The information displayed for each running flow is as follows:



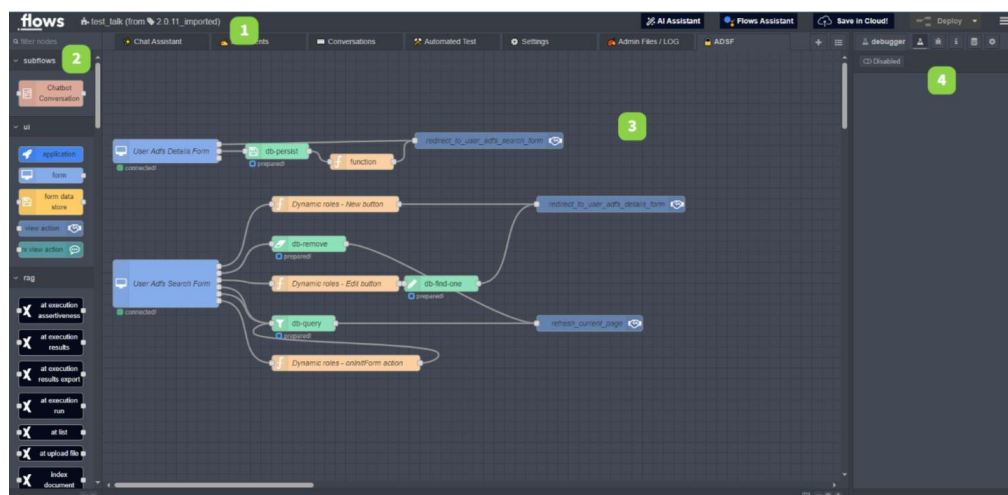
- **Name** of the development project.
- **Version running.** If the development project is up in "design mode," it means a new version of the development project is being designed/coded, starting from the version shown in the header. If the development project is up in "production mode," it means the version shown of the development project is being executed.
- Internally assigned name for the **Docker container** startup.
- Tag of the **Docker image** used at startup.
- **Container startup date.**
- **Volume bindings.**
- **Ports** occupied by the **flow designer**.
- **Ports for remote desktop connection** (VNC) to the Docker container.
- **Informative messages.**
 - If the remote desktop connection to the container has been enabled, the web will show an informative message with the VNC connection password to the Docker container.
 - If we have enabled the startup of the embedded Desktop Control Server within the Docker container itself, the system will display an informative message indicating this.

At the bottom of the card, we have the following buttons available:

- Real-time log visualization of the development project execution. Allows the internal logs of the **aXet.flows** execution to be viewed from the platform's web. This functionality enables the detection of possible internal anomalies presented by a development project, which go beyond the programming of the features themselves, and is an internal core error of the **aXet.flows** designer/executor.
- Download log file of the development project execution. If with the previous button, we could visualize the execution log, with this button we can download the log file.
- Stop the **aXet.flows** instance. Allows stopping the running instance, either in design mode or production mode on our PC.
- Remote desktop connection with HTTP client (noVNC) to the container. **aXet.flows**. Clicking this button will open a new tab in the web browser where we have remote desktop access to the Docker container itself, as if it were a physical computer with a desktop. On connecting to the desktop, we will be asked for the connection password. Once entered, we will have remote desktop access to open a bash and run commands inside the Docker container, and/or open Google Chrome, install packages, or whatever we consider necessary.
- Access the **aXet.flows** designer. When the **aXet.flows** is running in design mode, this button will open a new tab in the web browser, where we will have access to the **aXet.flows** designer.

Designer

Let's introduce the graphical interface of **aXet.flows** designer.



These are the points highlighted in the image:

1. The **header** at the top, containing the 'Deploy', 'AI Assistant', 'Flows Assistant' and 'Save in cloud' buttons, and main menu.

2. The **palette** on the left, containing the nodes available to use.

They are organised into a number of categories, with inputs, outputs and functions at the top. If there are any subflows, they appear in a category at the top of the palette.

Categories can be expanded or collapsed by clicking its header.

Above the palette is an input that can be used to filter the list of nodes.

3. The main **workspace** in the middle, where flows are developed by dragging nodes from the palette and wiring them together. The workspace has a row of tabs along the top; one for each flow and any subflows that have been opened.

The footer of the workspace contains buttons to zoom in and out as well as to reset to the default zoom level.

The workspace view can be customised via the 'View' tab of the User Settings dialog.

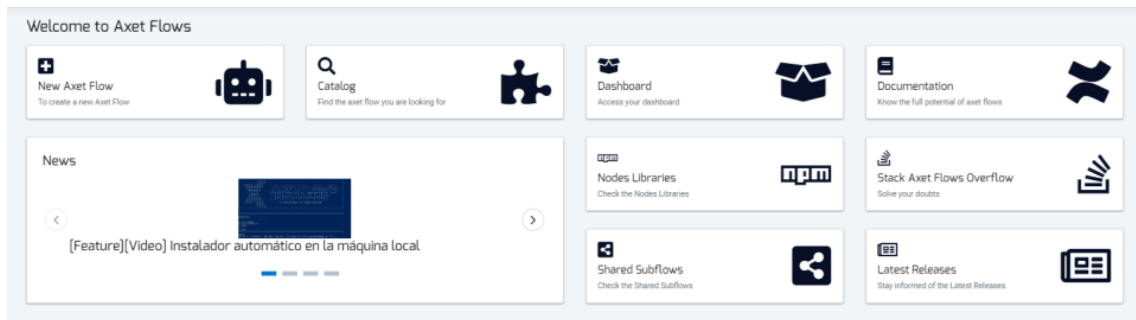
4. The **sidebar** on the right contains panels that provide a number of useful tools within the editor.
 - a. **Flow debugger.** It provides a flow debugger with the following features:
 - i. set breakpoints on node inputs or outputs
 - ii. pause the runtime manually or when a message arrives at a breakpoint
 - iii. once paused you can:
 1. inspect the queued up messages
 2. step forward individual messages
 3. drop messages
 - b. **Debug** sidebar displays messages passed to '*Debug nodes*' within the flow, as well as certain log messages from the runtime. The panel provides three options: all nodes, selected nodes, current flow.

- c. **Information** sidebar shows information about the flows. This includes an outline view of all flows and nodes, as well as details of the current selection.
- d. **Context data** sidebar displays the contents of the context data store. The panel is split into three sections, one for each context scope: node, flow and global.
- e. **Configuration Nodes** sidebar provides a list of all config nodes, organised by their scope. Each node shows its type and label, along with a count of how many regular flow nodes currently use that config node. If the config node is unused, it is shown with a dotted outline.

aXet.flows - Using it

Create a New Flow

On the Home Page of **aXet.flows**, you can find shortcuts to manage your development projects.



These are the features you can find:

- **New aXet flow.** Click on this button to create a new development project from scratch. You must complete the following fields:

Data

Name *

Main Project *

Use case

Category *

Use case *

Description

Description...

Search tags

- **Name** (It is a required field). It is the name that the development project will have.
- **Main project** (It is a required field). It is the project to which the development project will be associated. All users of the platform that have this project associated will have this development project visible.
- **Category** (It is a required field). Select from the dropdown the category that best fits the development.

- **Use case** (It is a required field). Once the category is selected, the use cases corresponding to that category are filled in. You must select a '*Use case*' for your development.
- **Description** (It is an optional field). It allows adding a description to the development project that can facilitate its search in the catalog and be reused by other users.
- **Search tags** (It is an optional field). It allows you to associate a series of keywords with the development project, so that later we can search for it by those keywords in the catalog.

Click on 'Save' button to create the development project. The platform will ask us if we want to manage the members who will have access to the new development project. Otherwise, we can manage the members later.

When we add a user as a member of a flow, we can assign them different types of permissions, to assign greater or lesser privilege in the manipulation of it. The types of permissions are as follows:

- **-Executor.** Members with this role will only be able to execute in "production mode," that is, to start and use a usable version at the end-user level of a development project. These users will not be able to design or evolve the development project in question.
- **-Forker.** In addition to the characteristics indicated for the Executor role, the ability to carry out a Fork of the flow in question is added.
- **-Modifier.** In addition to the characteristics indicated for the Forker role, the possibility of designing new versions of the development project is added, such that the development project could be blocked by this type of users to design/code new features or evolve the existing ones. Essentially, it is as if the source code were modified, at the level of development project design, of the development project to be able to release a new version of it.
- **-Co-owner.** A co-owner has permissions to use all available functionalities that can be carried out on a development project, except the possibility of deactivating it.

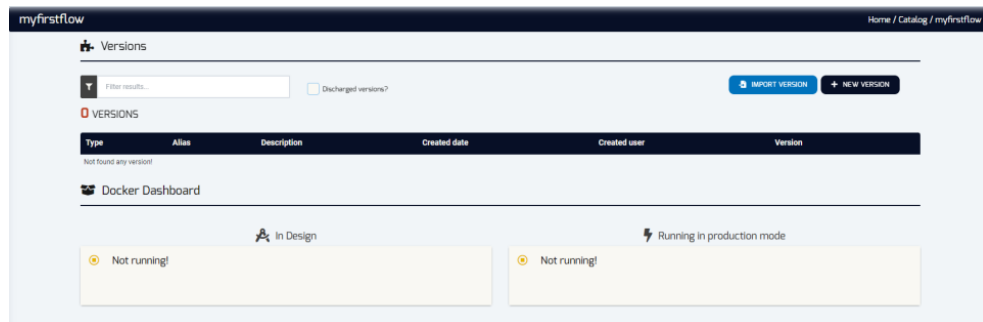
You can now find your development project in the **Catalog** to start developing it.

- **Catalog.** You can visualize all the information of the different development projects that, either you own them, or you have access to execute/evolve them.

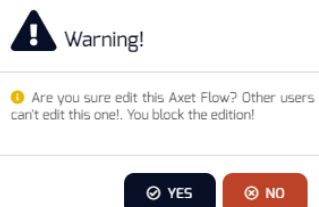
Select the development project you want to work on and click the '*Run this one!*' button.



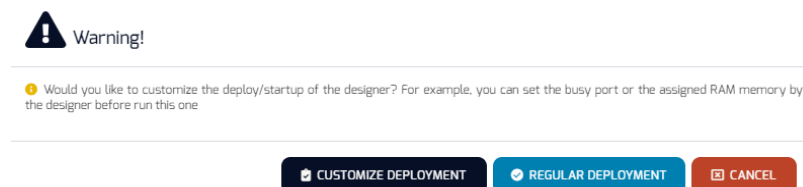
It will show you that there is currently no version of the development project available. Click on the 'New version' button.



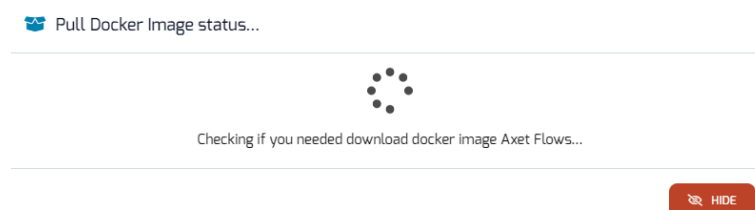
Click on 'Yes' in this Warning message.



Select 'Regular deployment' that will take you to the development environment to start designing or 'Customize deployment' if you want to work with an older version of the development environment.



The platform checks if it is necessary to update any of the components required to run the **axet.flows** designer.



Once the check is complete and the update, if needed, is done, the designer will open.

- **Dashboard.** This section allows for a global view of all development project instances that are operational/running on our PC.
- **Documentation.** This section takes you to the Confluence page where you can find information about **aXet.flows**.
- **Nodes Libraries.** This section takes you to Flow Repository / Nodes Libraries section
- **Stack aXet.flows Overflow.** This section takes you to a channel of **aXet.flows** platform so you can interact with other users.
- **Shared Subflows.** Shared flows should always be contextualized as subflows within the flow designer.

A **subflow** is a collection of nodes that are collapsed into a single node in the designer workspace.

They can be used to reduce some visual complexity of a flow, or to package up a group of nodes as a reusable flow used in multiple places.

Once created, the subflow is added to the palette of available nodes. Individual instances of the subflow can then be added to the workspace just like any other node.

This section shows the available shared subflows developed by **aXet.flows** users.

The screenshot displays the 'SHARED SUBFLOWS' section of the aXet.flows platform. It features a grid of subflow cards, each representing a reusable flow. Each card includes a title, version number, a brief description, and a 'Like' or 'Dislike' button. The cards are organized into a grid with columns for 'Project', 'Advanced Search', and 'Export'. A note at the top states: 'Shared subflows must be created inside a Axet Flows instance!'. Below this, it says '177 SHARED SUBFLOWS'. The cards shown include '371', 'Add extra hosts', 'AIDME Flow', 'API Global Params', 'API Params', and 'APIAuth'. On the right side, there is a 'Top 10' ranking list titled 'Most used' and 'Highest rated'. The 'Most used' list shows the following data:

Last version	Librarie	Used
2.2.0	Involvement	364
v4	Monitoring	307
0.0.1	uploader to buffer	236
v1.0.0	Send email with Wrapper	221
0.1	JIRA GET Request	198
v 1.1.0	convert-buffered-sip	195
v 1.2.1	Wok src	195
0.1.5	SP-Download	145
V.0.0.2	Send mail	103

The 'Highest rated' list shows the following data:

Last version	Librarie	Used
2.2.0	Involvement	9
V1.3	show Log & Settings	7
0.1.5	SP-Download	7
0.0.2	Easy Ute-register	6
v4	Monitoring	6
0.1.4	SP-Upload	6

You can click the 'Like' or 'Dislike' buttons to rate them. This way they will be highlighted in the ranking.

Shared subflows must be created inside a aXet.flows instance

- **Latest Releases.** This section takes you to a channel of **aXet.flows** platform so you can interact with other users.
- **News.** This section takes you to **aXet.flows** news.

Start a development project from scratch

You are going to create your first application with **aXet.flows** starting from scratch. You will start by getting to know the main elements available in the designer.

Nodes

The **palette** on the left contains the nodes available to use. You can find different sections:

- **ui.** You will find nodes that allow you to easily design the graphical interface of an application.
- **rag.** You will find nodes to implement a RAG (Retrieval Augmented Generation) system.
- **axet.** You will find nodes to use components of the **aXet** platform.
- **generativeai.** You will find nodes to use generative AI.
- **database.** You will find nodes to connect, create, and use a database.
- **network.** You will find nodes to use web services.
- **utils.** You will find nodes with different utilities.
- **parser.** You will find nodes to parse different file formats.
- **storage.** You will find nodes to work with files.
- **process control.** You will find nodes to monitor, debug, and manage errors in the application.
- **msg control.** You will find nodes to manage messages.
- **worker.** You will find nodes to manage '*Digital workers*'.
- **credentials.** You will find nodes to manage credentials in the application.
- **email.** You will find nodes to manage emails.
- **Excel Utils.** You will find nodes to manage Excel files.
- **session.** You will find nodes to manage sessions in the application.

- **audit.** You will find the ‘use case’ node, that is mandatory to inform the use cases that solve your development project.
- **enabler.** You will find AI nodes that allow, for example, transcribing and translating an audio file.

If you hover the mouse over the nodes found in these sections, you will see a description of them and know what you can use them for.

If you want to consult detailed information about each of the nodes available in the palette, select it in the workspace and choose the 'info' tab in the right sidebar.

Nodes can be added to the workspace by:

- Scroll through the palette until you find the node you want to use and drag it to the workspace.
- Use the search in the palette to filter the node you want to use and drag it to the workspace.
- Position yourself in the workspace, press the ‘CTRL’ key while clicking the left mouse button. A list of all available nodes will appear, select the one you want to use.

Here you can find documentation of the core set you should know about.

- **Inject.** The Inject node can be used to manual trigger a flow by clicking the node’s button within the editor. It can also be used to automatically trigger flows at regular intervals. The message sent by the **Inject** node can have its ‘payload’ and ‘topic’ properties set.
- **Debug.** The Debug node can be used to display messages in the Debug sidebar within the editor. The sidebar provides a structured view of the messages it is sent, making it easier to explore the message. Alongside each message, the debug sidebar includes information about the time the message was received and which **Debug** node sent it. Clicking on the source node id will reveal that node within the workspace.

The button on the node can be used to enable or disable its output. It is recommended to disable or remove any Debug nodes that are not being used.

The node can also be configured to send all messages to the runtime log, or to send short (32 characters) to the status text under the debug node.

- **Function.** The Function node allows JavaScript code to be run against the messages that are passed through it.

- **Change.** The Change node can be used to modify a message's properties and set context properties without having to resort to a Function node. Each node can be configured with multiple operations that are applied in order. The available operations are:
 - **Set** for set a property. The value can be a variety of different types, or can be taken from an existing message or context property.
 - **Change** for search and replace parts of a message property.
 - **Move** for move or rename a property.
 - **Delete** for delete a property.
- **Switch.** The Switch node allows messages to be routed to different branches of a flow by evaluating a set of rules against each message. The name "switch" comes from the "switch statement" that is common to many programming languages. The node is configured with the property to test - which can be either a message property or a context property. There are four types of rule:
 - **Value** rules are evaluated against the configured property.
 - **Sequence** rules can be used on message sequences, such as those generated by the Split node.
 - A JSONata **Expression** can be provided that will be evaluated against the whole message and will match if the expression returns a true value.
 - An **Otherwise** rule can be used to match if none of the preceding rules have matched.
- **Template.** The Template node can be used to generate text using a message's properties to fill out a template. If the template generates valid JSON or YAML content, it can be configured to parse the result to the corresponding JavaScript Object.

Nodes are wired together by pressing the left-mouse button on a node's port, dragging to the destination node and releasing the mouse button.

Messages

A flow works by passing messages between nodes. The messages are simple JavaScript objects that can have any set of properties.

Messages usually have a *'payload'* property - this is the default property that most nodes will work with.

The easiest way to understand the structure of a message is to pass it to a *'Debug node'* and view it in the *'Debug sidebar'*.

By default, the *'Debug node'* will display the **msg.payload** property, but can be configured to display any property or the whole message.

When displaying an Array or Object, the sidebar provides a structured view that can be used to explore the message.

- At the top it shows the name of the property that has been passed in.
- Next to the property name is the type of the property: Object, String, Array, etc.
- It then shows the contents of the property. For Arrays and Objects, the property is collapsed into a single line. By clicking on it, the property will expand to show more detail.

If a message property contains a JSON string it must first be parsed to its equivalent JavaScript object before the properties it contains can be accessed. To determine whether a property contains a String or Object, the Debug node can be used.

aXet.flows provides a JSON node to do this conversion.

A common task in a flow is to modify the properties of a message as it passes between nodes. For example, the result of an HTTP Request may be an object with many properties, of which only some are needed.

There are two main nodes for modifying a message, the Function node and the Change node.

- The Function node allows you to run any JavaScript code against the message. This gives you complete flexibility in what you do with the message, but does require familiarity with JavaScript and is unnecessary for many simple cases.
- The Change node provides a lot of functionality without needing to write JavaScript code. Not only can it modify message properties, but it can also access flow and global context.

Functions

The Function node allows JavaScript code to be run against the messages that are passed through it.

The message is passed in as an object called `msg`. By convention it will have a `'msg.payload'` property containing the body of the message.

Other nodes may attach their own properties to the message, and they should be described in their documentation.

Writing a Function

The code entered into the Function node represents the body of the function.

The function must always return a `msg` object. Returning a number or string will result in an error.

The returned message object does not need to be same object as was passed in; the function can construct a completely new object before returning it. For example:

```
var newMsg = { payload: msg.payload.length };  
return newMsg;
```

Use `node.warn()` to show warnings in the sidebar to help you debug. For example:

```
node.warn("my var xyz = " + xyz);
```

Multiple Outputs

The function edit dialog allows the number of outputs to be changed. If there is more than one output, an array of messages can be returned by the function to send to the outputs.

This makes it easy to write a function that sends the message to different outputs depending on some condition. For example, this function would send anything on topic “banana” to the second output rather than the first:

```
if (msg.topic === "banana") {  
  return [ null, msg ];  
} else {  
  return [ msg, null ];  
}
```

The following example passes the original message as-is on the first output and a message containing the payload length is passed to the second output:

```
var newMsg = { payload: msg.payload.length };  
return [msg, newMsg];
```

Multiple Messages

A function can return multiple messages on an output by returning an array of messages within the returned array. When multiple messages are returned for an output, subsequent nodes will receive the messages one at a time in the order they were returned.

In the following example, msg1, msg2, msg3 will be sent to the first output. msg4 will be sent to the second output.

```
var msg1 = { payload:"first out of output 1" };  
var msg2 = { payload:"second out of output 1" };  
var msg3 = { payload:"third out of output 1" };  
var msg4 = { payload:"only message from output 2" };  
return [ [ msg1, msg2, msg3 ], msg4 ];
```

The following example splits the received payload into individual words and returns a message for each of the words.

```
var outputMsgs = [];  
var words = msg.payload.split(" ");  
for (var w in words) {  
    outputMsgs.push({payload:words[w]});  
}  
return [ outputMsgs ];
```

Sending messages asynchronously

If the function needs to perform an asynchronous action before sending a message it cannot return the message at the end of the function.

Instead, it must make use of the node.send() function, passing in the message(s) to be sent. It takes the same arrangement of messages as that can be returned, as described in the previous sections.

For example:

```
doSomeAsyncWork(msg, function(result) {  
    msg.payload = result;  
    node.send(msg);  
})
```

```
});  
return;
```

The Function node will clone every message object you pass to `node.send` to ensure there is no unintended modification of message objects that get reused in the function.

The Function can request the runtime to not clone the first message passed to `node.send` by passing in `false` as a second argument to the function. It would do this if the message contains something that is not otherwise cloneable, or for performance reasons to minimise the overhead of sending messages:

```
node.send(msg, false);
```

Finishing with a message

If a Function node does asynchronous work with a message, the runtime will not automatically know when it has finished handling the message.

To help it do so, the Function node should call `node.done()` at the appropriate time. This will allow the runtime to properly track messages through the system.

```
doSomeAsyncWork(msg, function(result) {  
    msg.payload = result;  
    node.send(msg);  
    node.done();  
});  
return;
```

Running code on start

The Function node provides an On Start tab where you can provide code that will run whenever the node is started. This can be used to setup any state the Function node requires.

For example, it can initialise values in local context that the main Function will use:

```
if (context.get("counter") === undefined) {  
    context.set("counter", 0)  
}
```

The On Start function can return a Promise if it needs to complete asynchronous work before the main Function can start processing messages. Any messages that arrive before the On Start function has completed will be queued up, and handled when it is ready.

Tidying up

If you do use asynchronous callback code in your functions then you may need to tidy up any outstanding requests, or close any connections, whenever the flow gets re-deployed. You can do this in two different ways.

Either by adding a close event handler:

```
node.on('close', function() {  
    // tidy up any async code here - shutdown connections and so  
    on.  
});
```

Or, you can add code to the On Stop tab in the node's edit dialog.

Logging events

If a node needs to log something to the console, it can use one of the follow functions:

```
node.log("Something happened");  
node.warn("Something happened you should know about");  
node.error("Oh no, something bad happened");
```

Where the console output appears will depend on how your operating system and how you start **aXet.flows**. If you start using a command line - that is the console where logging will appear. If you run as a system service then it may appear in the system log. If you run under an app like PM2 it will have it's own way for showing logs.

The warn and error messages also get sent to the debug tab on the right side of the flow editor.

For finer grained logging, `node.trace()` and `node.debug()` are also available. If there is no logger configured to capture those levels, they will not be seen.

Handling errors

If the function encounters an error that should halt the current flow, it should return nothing. To trigger a Catch node on the same tab, the function should call `node.error` with the original message as a second argument:

```
node.error("hit an error", msg);
```

Storing data

Aside from the `msg` object, the function can also store data in the context store.

In the Function node there are three predefined variables that can be used to access context:

- context - the node's local context
- flow - the flow scope context
- global - the global scope context

The following examples use flow context, but apply equally well to context and global.

There are two modes for accessing context; either synchronous or asynchronous. The built-in context stores provide both modes. Some stores may only provide asynchronous access and will throw an error if they are accessed synchronously.

To get a value from context:

```
var myCount = flow.get("count");
```

To set a value:

```
flow.set("count", 123);
```

The following example maintains a count of how many times the function has been run:

```
// initialise the counter to 0 if it doesn't exist already
var count = context.get('count') || 0;
count += 1;
// store the value back
context.set('count', count);
// make it part of the outgoing msg object
msg.count = count;
return msg;
```

Get/Set multiple values

It is also possible to get or set multiple values in one go:

```
var values = flow.get(["count", "colour", "temperature"]);
// values[0] is the 'count' value
// values[1] is the 'colour' value
// values[2] is the 'temperature' value

flow.set(["count", "colour", "temperature"], [123, "red",
"12.5"]);
```

In this case, any missing values are set to null.

Asynchronous context access

If the context store requires asynchronous access, the get and set functions require an extra callback parameter.

```
// Get single value
flow.get("count", function(err, myCount) { ... });

// Get multiple values
flow.get(["count", "colour"], function(err, count, colour) { ...
})

// Set single value
flow.set("count", 123, function(err) { ... })

// Set multiple values
flow.set(["count", "colour"], [123, "red"], function(err) { ...
})
```

The first argument passed to the callback, err, is only set if an error occurred when accessing context.

The asynchronous version of the count example becomes:

```
context.get('count', function(err, count) {
  if (err) {
    node.error(err, msg);
  } else {
    // initialise the counter to 0 if it doesn't exist
    already
    count = count || 0;
    count += 1;
    // store the value back
    context.set('count', count, function(err) {
      if (err) {
        node.error(err, msg);
      } else {
        // make it part of the outgoing msg object
        msg.count = count;
        // send the message
        node.send(msg);
      }
    });
  }
});
```

```
    }  
  });
```

Multiple context stores

The get/set context functions accept an optional parameter to identify the store to use.

```
// Get value - sync  
var myCount = flow.get("count", storeName);  
  
// Get value - async  
flow.get("count", storeName, function(err, myCount) { ... });  
  
// Set value - sync  
flow.set("count", 123, storeName);  
  
// Set value - async  
flow.set("count", 123, storeName, function(err) { ... })
```

Global context

The global context can be pre-populated with objects when **aXet.flows** starts. This is defined in the main settings.js file under the functionGlobalContext property.

This can be used to load additional modules within the Function node.

Adding Status

The function node can also provide it's own status decoration in the same way that other nodes can. To set the status, call the node.status function. For example:

```
node.status({fill:"red",shape:"ring",text:"disconnected"});  
node.status({fill:"green",shape:"dot",text:"connected"});  
node.status({text:"Just text status"});  
node.status({}); // to clear the status
```

Any status updates can then also be caught by the Status node.

Loading additional modules

Additional node modules cannot be loaded directly within a Function node. They must be loaded in your settings.js file and added to the functionGlobalContext property.

For example, the built-in `os` module can be made available to all functions by adding the following to your `settings.js` file.

```
functionGlobalContext: {  
  osModule:require('os')  
}
```

at which point, the module can be referenced within a function as `global.get('osModule')`.

Modules loaded from your settings file must be installed in the same directory as the settings file.

By setting `functionExternalModules` to `true` in your `settings.js` file, the Function node's edit dialog will provide a list where you can add additional modules that should be available to the node. You also specify the variable that will be used to refer to the module in the node's code.

Handling a Timeout

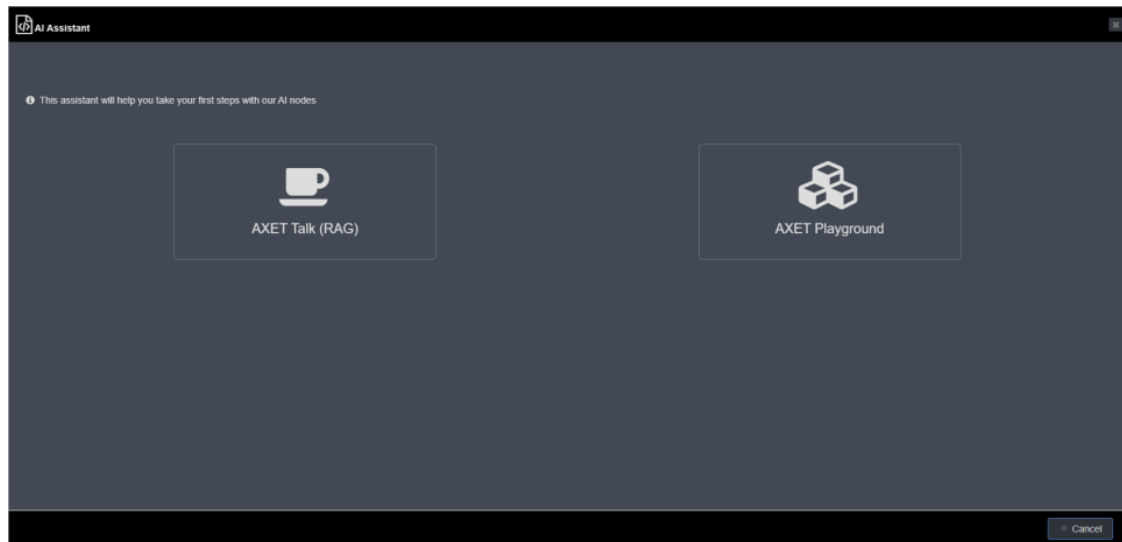
It is possible to set a timeout for the function node on the Setup tab. This value, in seconds, is how long the runtime will allow the Function node to run for before raising an error. If set to 0, the default, no timeout is applied.

In case none of the nodes of the tool explicitly provide the functionality you need, and additionally this is so limited that it is not worth developing new nodes, you can lower the level of abstraction that the tool provides by default and directly code the expected behavior in JavaScript, in a simple manner. For this, you will have available the 'msg' object which will contain all the flow information at the execution step we are at the moment of execution.

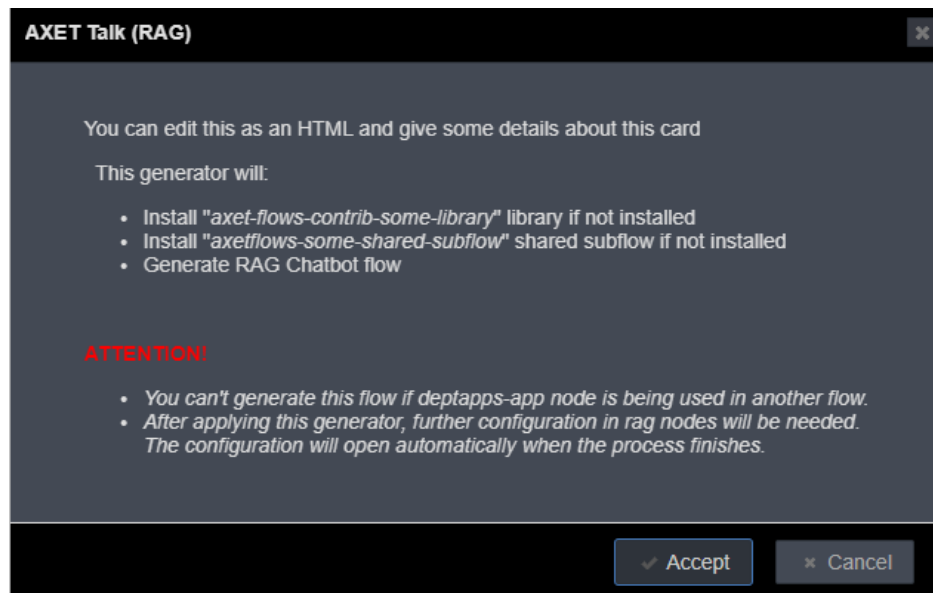
AI Assistant

This is the AI assistant from **aXet.flows**.

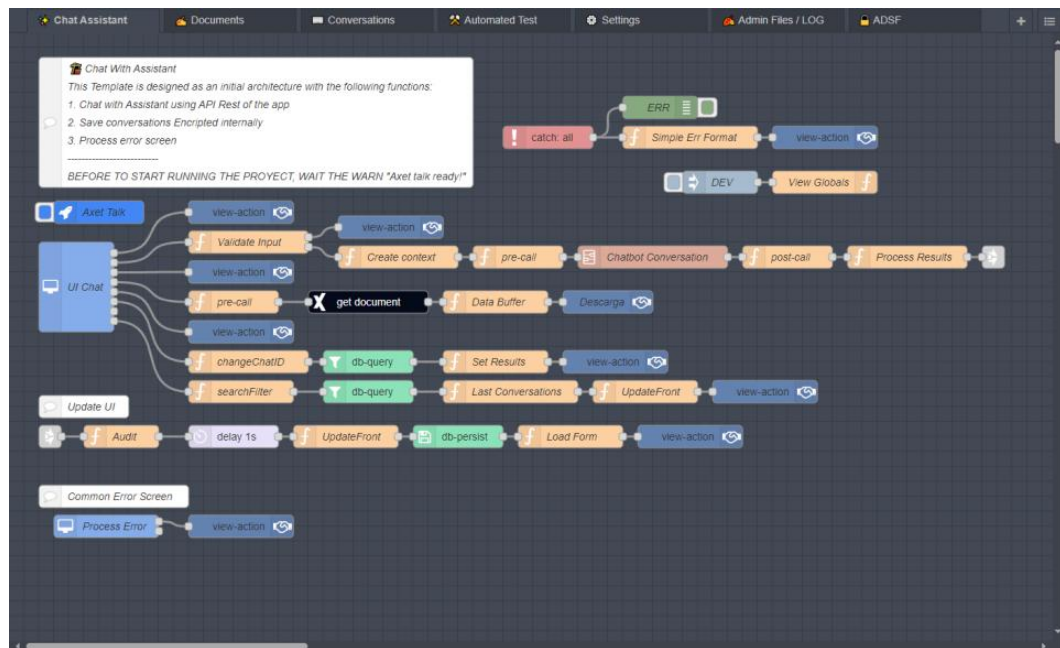
When you start it, a window opens with two options: **aXet.talk** and **aXet.playground**.



- **aXet.talk.** By selecting this option, you will get an application to which you just have to add the credential configuration for it to be operational.

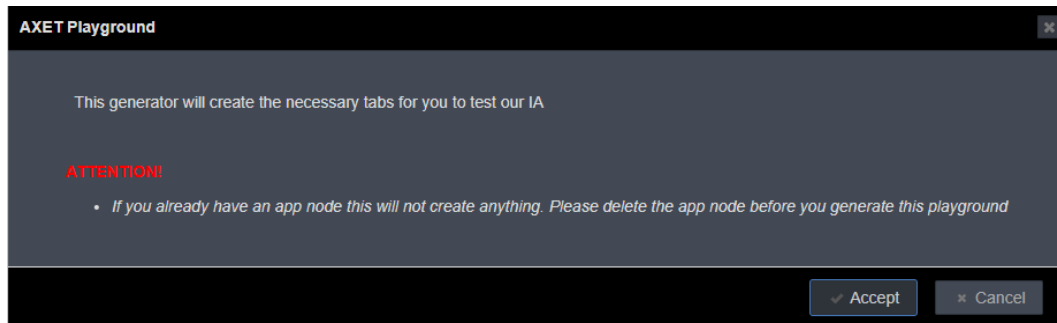


Once the assistant has been executed, you can review the automatically created nodes.

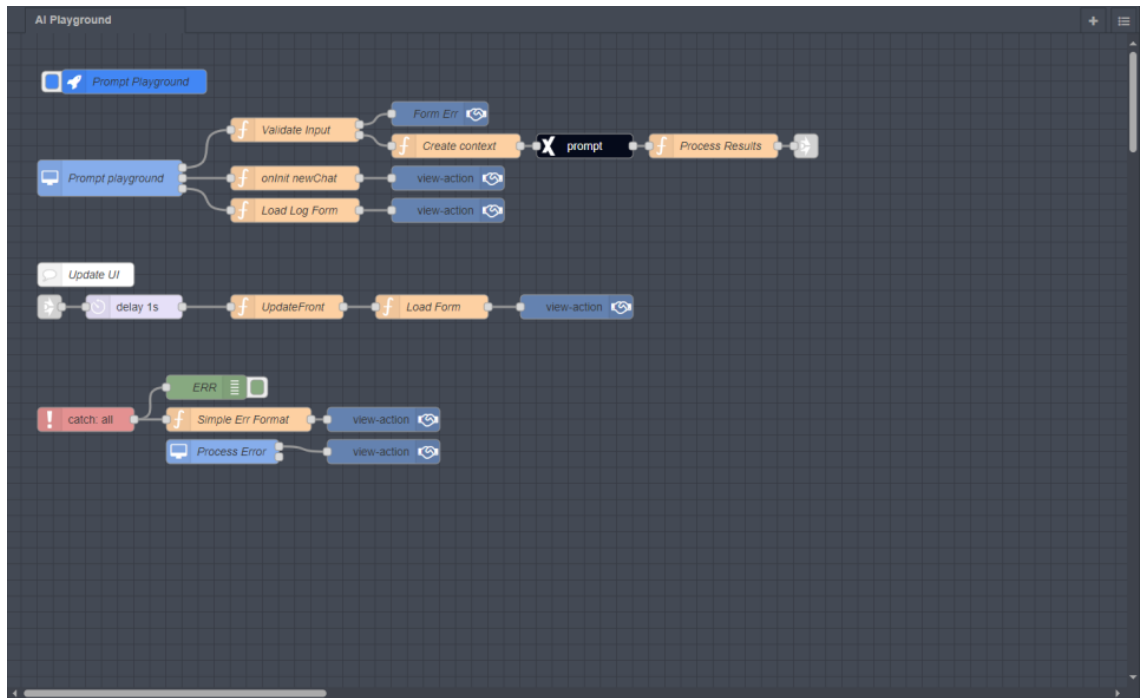


After finishing implementing the application, you can run it clicking on the ‘Deploy’ button.

- **aXet.playground.** By selecting this option, you will obtain an application that will allow you to make inquiries to the AI, once you have connected to **aXet.enabler**.



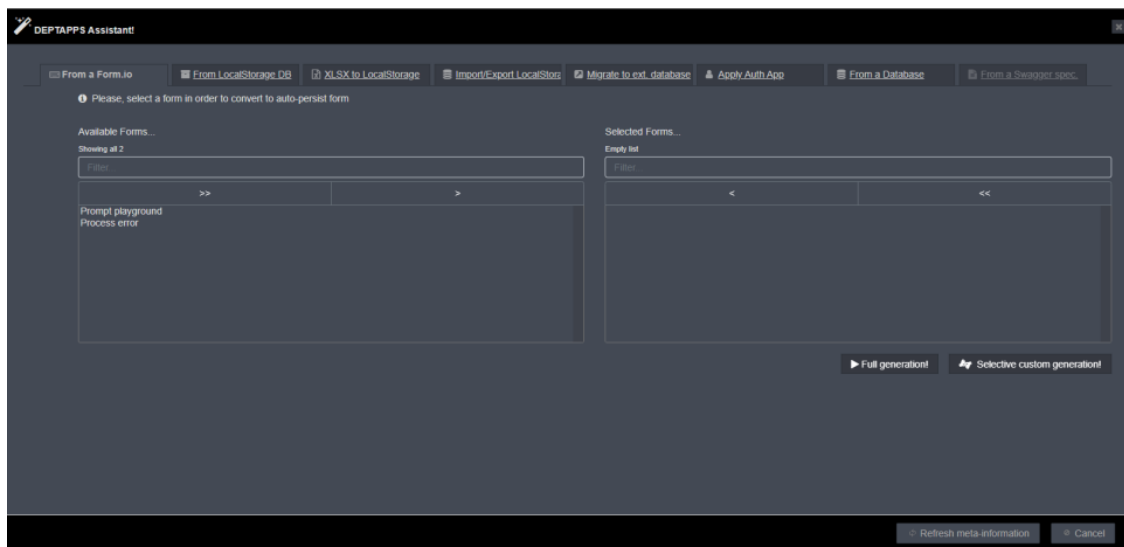
Once the assistant has been executed, you can review the automatically created nodes.



After finishing implementing the application, you can run it clicking on the ‘Deploy’ button.

Flows Assistant

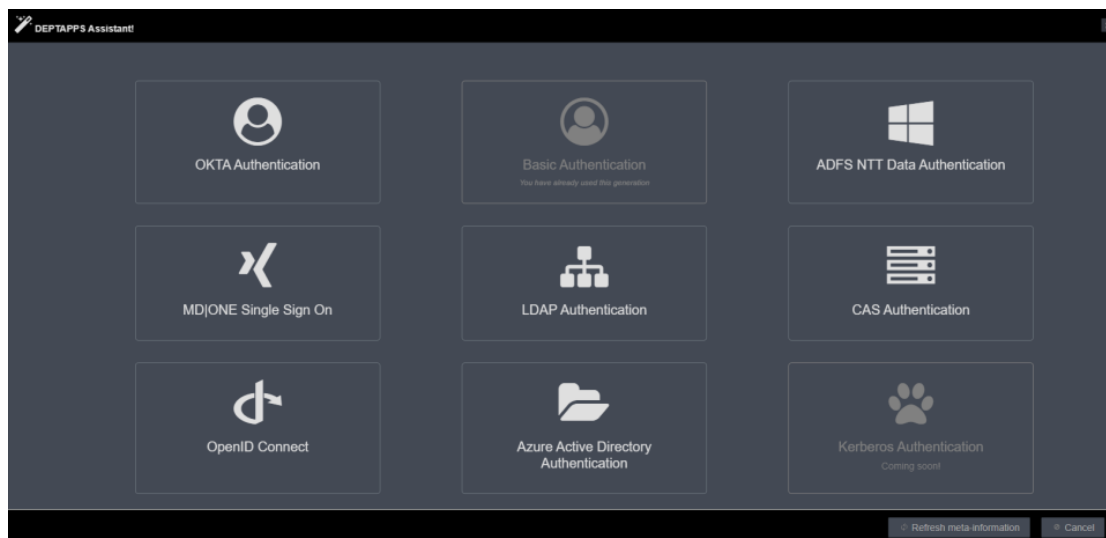
The assistants you can find in this section will help you generate flows quickly.



These are the available options:

- **From a Form.io.** Allows you to create a database, and a search and CRUD system from a form designed in **aXet.flows**.

- **From LocalStorage DB.** Allows the creation of a CRUD maintenance system and RESTful API endpoints from a database designed in **aXet.flows**.
- **XLSX to LocalStorage.** Allows you to create a database from an Excel file in XLSX format. Create a database for each sheet of the Excel file.
- **Import/Export LocalStorage.** Since the databases of a development project are encrypted so that only it can use them, this assistant allows transferring the databases from one development project to another.
- **Migrate to ext. database.** Allows exporting the internal databases used by an application designed with **aXet.flows** to an external non-SQL database like MongoDB or Cassandra.
- **Apply Auth App.** Allows you to automatically generate an authentication layer for your application. You have different options available.



OKTA Authentication. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use *OKTA Authentication Login* type.

Configure your app site-map with a new administrator area, with user management form.

Basic Authentication. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Generate a default administrator user (*admin/admin*)

Configure your app to use *basic authentication login* type.

Configure your app site-map with a new administrator area, with user management form.

ADFS NTT Data Authentication. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use ADFS NTT Data authentication login type.

Configure your app site-map with a new administrator area, with user management form.

MD|ONE Single Sign On. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use ADFS NTT Data authentication login type.

Configure your app site-map with a new administrator area, with user management form.

LDAP Authentication. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use LDAP authentication login type.

Configure your app site-map with a new administrator area, with user management form.

After apply this authentication method, is necessary to configure the LDAP Domains on the application node. The configuration will open automatically when the process finish.

CAS Authentication. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use CAS authentication login type.

Configure your app site-map with a new administrator area, with user management form.

After apply this authentication method, is necessary to configure the CAS URL. The configuration will open automatically when the process finish.

OpenID Connect. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use OpenID authentication login type.

Configure your app site-map with a new administrator area, with user management form.

After applying this authentication method, you need to configure some parameters on the application node. The configuration will open automatically when the process finish.

Azure Active Directory Authentication. This authentication method will:

Generate a local-storage database for persist users

Generate CRUD forms for manage users.

Configure your app to use Azure AD authentication login type.

Configure your app site-map with a new administrator area, with user management form.

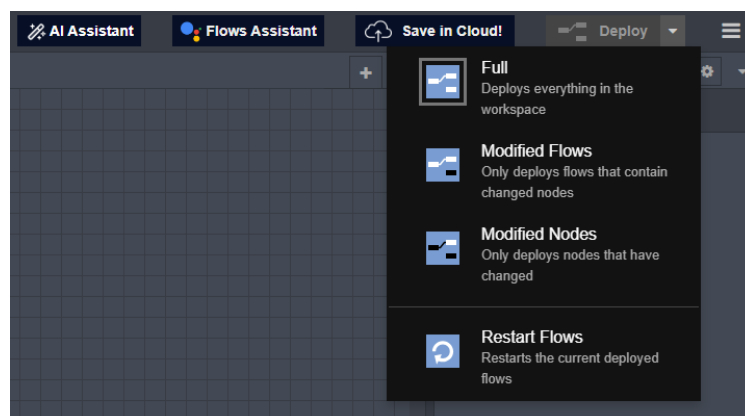
After applying this authentication method, you need to configure some parameters on the application node. The configuration will open automatically when the process finish.

- **From a Database.** If your application has meta-information related to the database it uses, this assistant allows you to automatically generate a database maintenance system.

After finishing implementing the application, you can run it clicking on the '*Deploy*' button.

Deploy

You have finished designing your flow, now you can deploy it. Click on the '*Deploy*' button.



You can find three options to deploy your flow:

- **Full.** Deploys everything in the workspace.

If you select this option currently running flows are interrupted and restarted.

- **Modified Flows.** Only deploys flows that contain changed nodes.
- **Modified Nodes.** Only deploy nodes that have changed.

The last two options can be really handy when you have different flows spread across your workspace or tabs but you don't want to reload them all each time you deploy. This way, nodes and/or flows that haven't been modified will continue to run without interruption.

You can find also the option to **Restart Flows**, that restart the current deployed flows. This means that currently running flows are interrupted and restarted.

When a message informs you that the process has been successfully completed, you can run the application.

Save in Cloud

Once you have developed your flows for your development project, it is time to collaboratively store it in the **aXet.flows** cloud. To do this, click on the 'Save in Cloud!' button and a popup like the following image will appear:

A screenshot of a dark-themed dialog box titled "Create new version of this DEPTAPP!". The dialog has a close button (X) in the top right corner. It contains two main input fields: "Version alias" with a text input box and a placeholder "Version alias...", and "Description message" with a larger text area and a placeholder "Description message...". Below the "Version alias" field, there is a small note: "(*) The last version saved is aliased to '1.0.1'". Below the "Description message" field, there is another note: "(*) The changes to be stored in the cloud correspond to the last deploy made. Keep it in mind!". At the bottom of the dialog, there are two buttons: "Save" and "Cancel".

You need to enter the required information to create a new version of the development project:

- **Version alias** (the use of XYZ versioning rules is recommended).

- **Version description.**

Once this is done, by clicking the 'Save' button, the designer will communicate with **aXet.flows** cloud to store all the information of the coding flows of the development project. In this way, a new version of the software associated with the development project is stored, which can be reproduced and executed on any other PC that has the **aXet.flows** platform installed.

This way, we achieve collaborative management/execution of development projects. The software is developed by one or more actors and can also be executed/used by one or more end users.

Production Deployment

You have already developed your application using the nodes from the palette and the assistants provided by **aXet.flows**, you have deployed it locally and managed versions of the development project. Now it's time to deploy the application in production.

With **aXet.flows** there are two ways to deploy your application in production, **cloud** and **standalone**. You must take into account:

- If there is no 'app' node in your application's flow, you will only be able to deploy it in standalone mode.
- If you want to deploy your application in the cloud, you must include an 'app' node in its design.
- If you have used an 'app' node in the design of your application and this 'app' node also uses OKTA as a login method, you will be able to deploy the application in cloud and standalone mode.

Select the development project from the Catalog and click '*Run this one!*' to be able to deploy in production. Then, click on *Deploy* icon, as you can see in the image bellow, to start the production deployment.



This action opens a new window to launch the required analysis of your project.



Before deploy your application in production you must pass the following analysis:

- **Audit Status.** Click on '*Request audit*' to launch the audit of your development project. An **aXet.flows** team will audit the project to ensure that nothing improper is being done, nor is there any unregistered use of AI.

Furthermore, it will verify that the use cases selected when registering the development project are appropriate and properly informed in the flows.

Remember that you must use the '*use case*' nodes to inform the use cases that resolve each of the flows you have designed in your development project.

The validity of a successfully passed audit is 6 months, in any case, the user can see the validity date in the same window.

- **Security Analysis Status.** Click on '*Scan*' to launch the security analysis that will verify that the project has the necessary security conditions to be published.

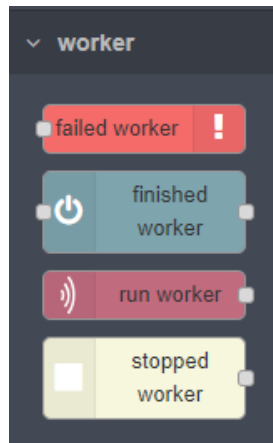
If either of the two analyses are not successfully passed, **aXet.flows** will inform you of the issues found and how to resolve them. Once corrected, you will need to run the analyses again.

With the two analyses executed and successfully passed, you can deploy your application in production.

Digital Workers

You have already seen how to develop web applications and desktop applications using **aXet.flows**, and that you have nodes in the palette that allow you to create the interface for these applications.

But if what you need is to develop an automation without an interface to run it unattended, **aXet.flows** has a category of nodes that will help you create it. This category is '*Worker*'.



If you want to consult detailed information about each of the nodes available in the palette, select it in the workspace and choose the 'info' tab in the right sidebar.

Add a 'Run Worker' node to your flow to be able to execute it unattended.

To start/stop the execution of your application, you must go to the main page of **aXet.flows** and select the 'Digital Workers' section. Here you will find all your development projects that use 'Worker' nodes with information about their status, buttons to start or stop their execution, and log consultation.

Flows Repository

In this menu you can find three items: **Nodes Libraries**, **Shared Subflows** and **All Rankings**. You will see each one of them.

Nodes Libraries

This section shows the available Nodes Libraries developed by **aXet.flows** users.

The screenshot shows the DEPTAPPS Nodes Libraries interface. At the top, there's a navigation bar with 'NODES LIBRARIES' and 'Home / Nodes Libraries (Private NPM Registry)'. Below this is a search filter section with dropdowns for 'Last Update', 'Project', and a 'Filter results...' input, along with an 'EXPORT' button and a 'Show discharge?' checkbox. The main area displays a grid of library cards. Each card shows the library name, version, creation and last modification dates, and a list of associated projects. On the right, there are two tables: 'Most used' and 'Highest rated', both showing a list of libraries with their versions and usage counts.

On the right panel you can find the ‘Most Used’ and the ‘Highest Rated’ Nodes Libraries.

By default, you see all the Nodes Libraries, but if you are looking for one, you can filter by last update, by project or by name, as you can see at the top of the image.

This is the information you can find in each of the node libraries:

- **Name**, in the header of the card.
IMPORTANT: The name of all libraries must have the name "deptapps-flows-contrib-*". If another name is given to the library in the package.json file, the DEPTAPPS platform will detect incorrect use of the archetype and will not allow the library to be loaded.
- **MD|ONE project** assigned to **aXet.flows**.
- **Associated MD|ONE projects**. These are additional projects to the previous one, whose users will be able to use the library, but will not be able to upload new artifacts of the library, that is, they will not be able to evolve it.
- **Creation date**.
- **Last modification date**.
- **Last user who modified the library**.

These are the actions you can perform with the selected library:

- **Like**. Allows you to like the library in question, with the aim of positive rating for community type libraries enabling other users to view the library usage and reputation at a glance.

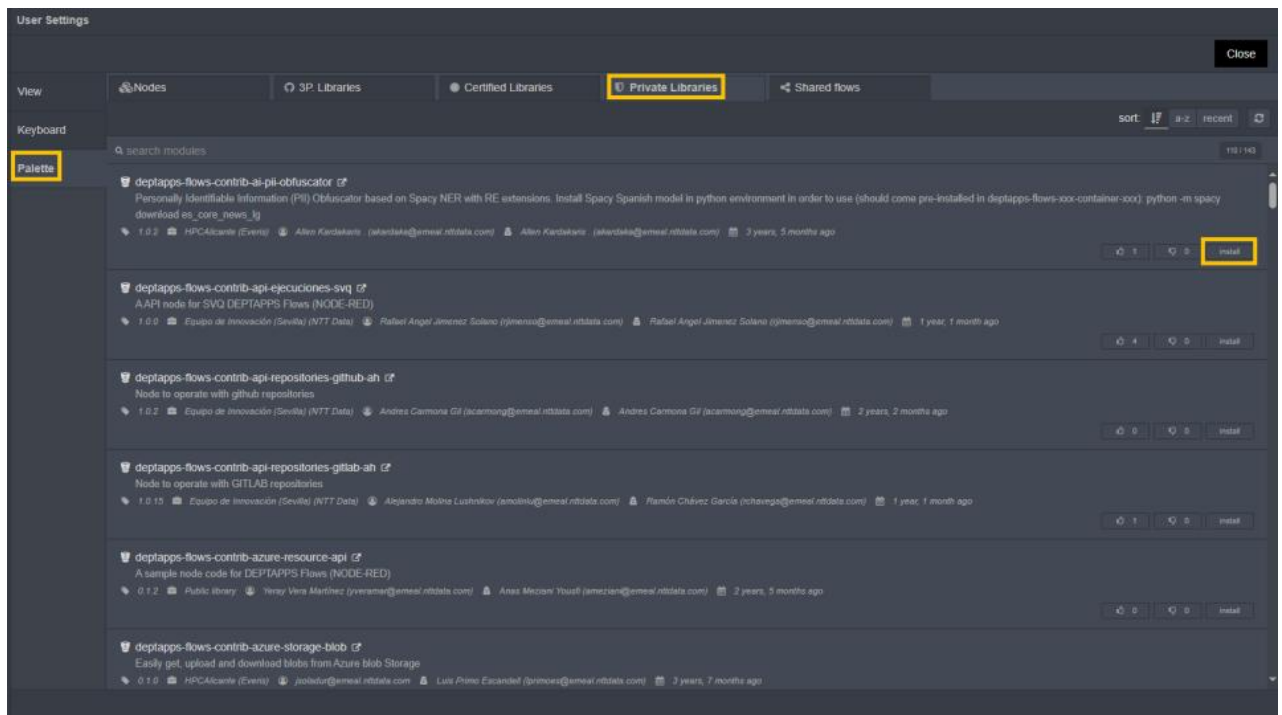
- **Dislike.** Allows you to like the library in question, with the aim of negative rating for community type libraries enabling other users to view the library usage and reputation at a glance.
- **Delete.** Allows for a logical deletion of the library in question.
- **Edit.** Allows you to edit a library. The following data can be edited:
 - URL of the source code repository where the library is stored.
 - MD|ONE project associated with the library.
 - Description.
 - Upload a new version of the library packaging, with an update of the source code. IMPORTANT: To upload new versions, it is very important to modify the version field of the library's package.json file. If an existing or previously used version is used, **aXet.flows** website will not allow the upload of the node library.
- Access to the library in the private NPM Registry of **aXet.flows** where the libraries are being managed.
- Access to the source code repository.

Click on 'Export' button to download the result of your search in Excel file format.

Nodes libraries are created in aXet.flows designer

To use a **Node Library** in the **designer**, follow these steps:

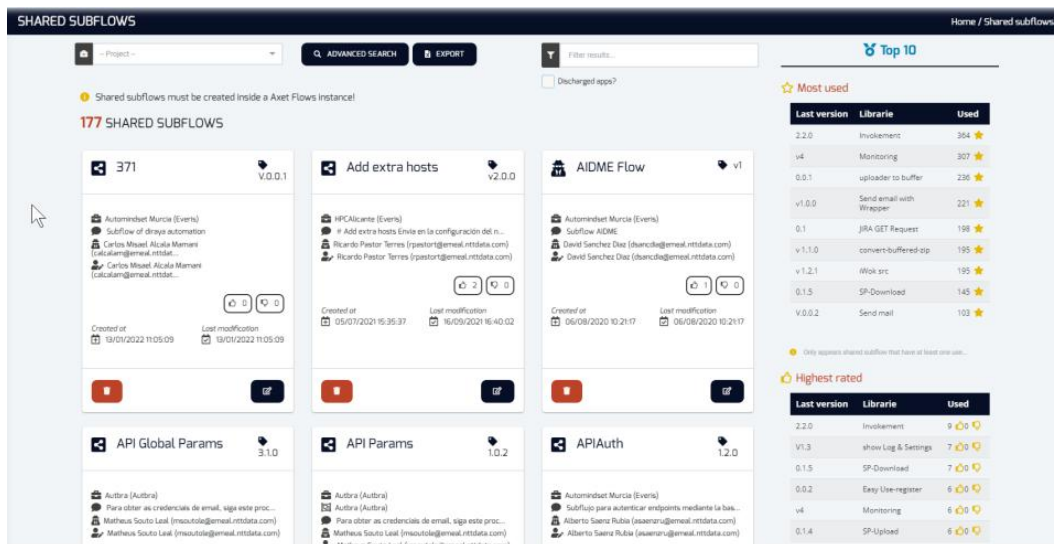
1. Expand the menu in the upper right corner.
2. Select the 'Settings' item.
3. In the window that has opened, select 'Palette' and the 'Private Libraries' tab.
4. Look for the node library you are interested in using in your flow and click on the 'Install' button.



5. You have the node library available in the palette.

Shared Subflows

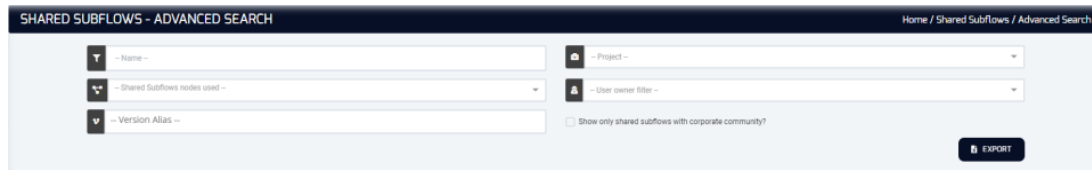
This section shows the available Shared subflows developed by **aXet.flows** users.



On the right panel you can find the *'Most Used'* and the *'Highest Rated'* Shared subflows.

By default, you see all the Shared subflows, but if you are looking for one, you can filter by project or by name, as you can see at the top of the image.

You can also click on '*Advanced Search*' button and a form will open for you to refine the search.



The actions you can carry out on the '*Subflows*' are as follows:

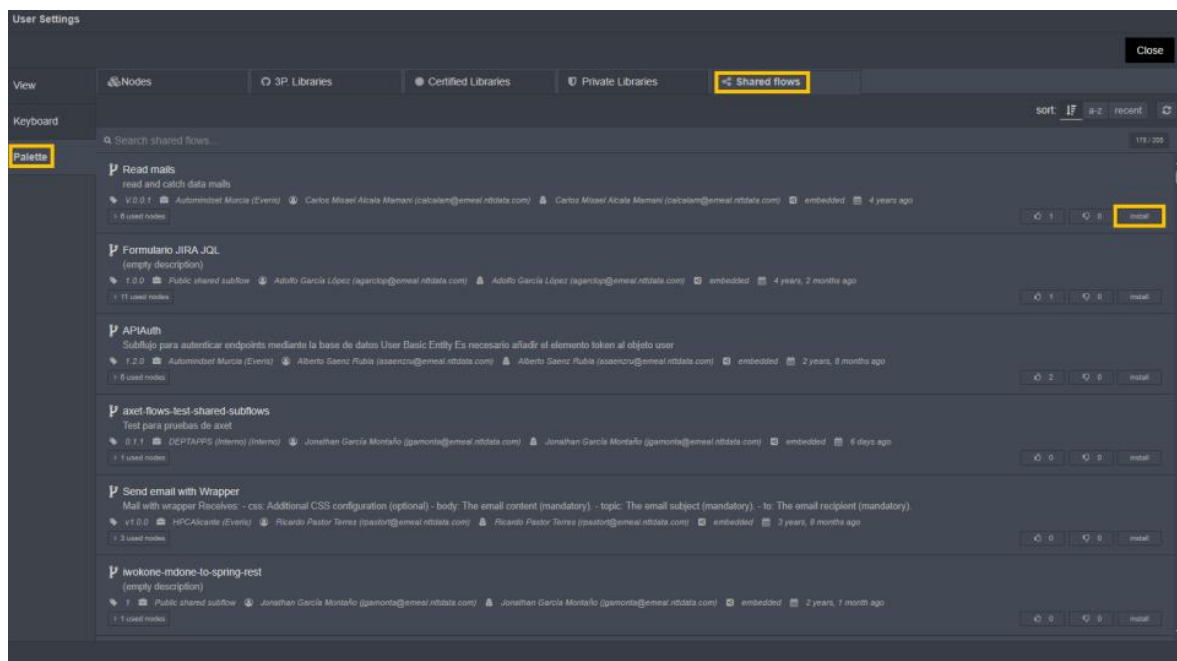
- **Like.** Allows you to give '*Like*' to the flow in question, with the aim that the shared flows that are of community type which can be used by any member of the **aXet.flows** community, can be positively rated, so that other users can see at a glance the usage of the shared flow, as well as its reputation.
- **Dislike.** Allows you to give '*Dislike*' to the flow in question, with the aim that the shared flows that are of community type which can be used by any member of the **aXet.flows** community, can be negatively rated, so that other users can see at a glance the usage of the shared flow, as well as its reputation.
- **Delete.** Allows you to perform a logical deletion of the (sub)flow in question.
- **Edit.** Allows you to edit a (sub)flow. Among the data that can be edited are the following:
 - Name of the subflow.
 - MD|ONE project to which it is associated.
 - Description of the subflow.
 - The versions tab will simply allow us to view the different versions of the subflow that have been published on **aXet.flows**.

Click on '*Export*' button to download the result of your search in Excel file format.

Shared subflows must be created inside a aXet.flows instance

To use a **Shared Subflow** in the **designer**, follow these steps:

1. Expand the menu in the upper right corner.
2. Select the '*Settings*' item.
3. In the window that has opened, select '*Palette*' and the '*Shared flows*' tab.
4. Look for the shared flow you are interested in using in your flow and click on the '*Install*' button.



5. You have the shared flow available in the palette.

All Rankings

In this section, you can find the most used and highly rated node libraries and shared subflows by **aXet.flows** users. Select each of the four tabs to view the listings.

RANKINGS
Home / All Rankings

Nodes Libraries Most used
Nodes Libraries Highest rated
Shared subflows Most used
Shared subflows Highest rated

1
deptapps-flows-contrib-email
Francisco Javier Romero Yesares (fromero@emeal.ntdata.com)
jsoladur@emeal.ntdata.com
2482 ★
1.1.17

2
deptapps-flows-contrib-sharepoint-client
Anas Meziani Youfi (ameziani@emeal.ntdata.com)
avizute@emeal.ntdata.com
1111 ★
0.4.13

3
deptapps-flows-contrib-exa-templating
mnavapad@emeal.ntdata.com
avizute@emeal.ntdata.com
1050 ★
1.11.9

4
deptapps-flows-contrib-excel-utils
Francisco Javier Romero Yesares (fromero@emeal.ntdata.com)
avizute@emeal.ntdata.com
678 ★
1.0.2

Private Libraries most used

# N°	Last version	Librarie	Axet Flows	Description	User owner	Last update user	Used
1	1.1.17	deptapps-flows-contrib-email	Public Library	A node to send and receive simple e...	jsoladur@emeal.ntdata.com	Francisco Javier Romero Yesares (fr...	2482 ★
2	0.4.13	deptapps-flows-contrib-sharepoint-client	Public Library	Take it easy with the High-Level Sh...	avizute@emeal.ntdata.com	Anas Meziani Youfi (ameziani@emeal...	1111 ★
3	1.11.9	deptapps-flows-contrib-exa-templating	Public Library	Templating engine based on Handleba...	avizute@emeal.ntdata.com	mnavapad@emeal.ntdata.com	1050 ★
4	1.0.2	deptapps-flows-contrib-excel-utils	HPCAlicante (Everbis)	Nodes to take it easy with Excel an...	avizute@emeal.ntdata.com	Francisco Javier Romero Yesares (fr...	678 ★
5	1.0.25	deptapps-flows-contrib-edit-excel	Public Library	Node to modify xlsx files for DEPTA...	mnavapad@emeal.ntdata.com	Jonathan Garcia Montaño (jgamenta@e...	603 ★
6	1.0.0	deptapps-flows-contrib-credentials	HPCAlicante (Everbis)	Add credentials to your flow in a s...	avizute@emeal.ntdata.com	Francisco Javier Romero Yesares (fr...	579 ★
7	1.5.2	deptapps-flows-contrib-nodes-map-to-xlsx	Web Corporativa (ibendrola) (ibendr...	Map object properties to excel cell...	Victor Gonzalez Fernandez (vgonzafe...	Victor Gonzalez Fernandez (vgonzafe...	487 ★
8	0.0.15	deptapps-flows-contrib-selenium-wd2	Public Library	A selenium node for browser automat...	mnavapad@emeal.ntdata.com	Yeray Vera Martinez (yveramar@emeal...	395 ★
9	0.6.9	deptapps-flows-contrib-jira-jql	Public Library	A node for jira management for DEPT...	mnavapad@emeal.ntdata.com	Anas Meziani Youfi (ameziani@emeal...	392 ★
10	0.2.3	deptapps-flows-contrib-plugin-ivokone	Public Library	A sample node code for DEPTAPPS Ro...	bpenasan@emeal.ntdata.com	rguerga@emeal.ntdata.com	282 ★
11	0.0.1	deptapps-flows-contrib-zip	Automindset Murcia (Everbis)	Node to compress or decompress zip...	avizute@emeal.ntdata.com	avizute@emeal.ntdata.com	240 ★
12	1.4.0	deptapps-flows-contrib-puppeteer	HPCAlicante (Everbis)	DEPRECATED, use a node function ins...	avizute@emeal.ntdata.com	avizute@emeal.ntdata.com	233 ★
13	0.1.4	deptapps-flows-contrib-oracle-ng	Public Library	This library must be used with DEPT...	jsoladur@emeal.ntdata.com	Jonathan Garcia Montaño (jgamenta@e...	196 ★

Nodes Libraries

By using this functionality, you can create your own nodes and thus extend the possibilities offered by **aXet.flows**.

Creating Nodes

There are some general principles to follow when creating new nodes. These reflect the approach taken by the core nodes and help provided a consistent user-experience.

Nodes should:

- **Be well-defined in their purpose.** A node that exposes every possible option of an API is potentially less useful than a group of nodes that each serve a single purpose.
- **Be simple to use, regardless of the underlying functionality.** Hide complexity and avoid the use of jargon or domain-specific knowledge.
- **Be forgiving in what types of message properties it accepts.** Message properties can be strings, numbers, booleans, buffers, objects, arrays or nulls. A node should do The Right Thing when faced with any of these.
- **Be consistent in what they send.** Nodes should document what properties they add to messages, and they should be consistent and predictable in their behaviour.
- **Catch errors.** If a node throws an uncaught error, **aXet.flows** will stop the entire flow as the state of the system is no longer known.

Wherever possible, nodes must catch errors or register error handlers for any asynchronous calls they make.

To implement your own nodes, you must build a NodeJS project. These nodes must comply with the archetype that you will find at this [link](#).

Nodes consist of a pair of files:

- A JavaScript file that defines the runtime behavior of the node.
- An html file defines how the node appears with the editor. It contains three distinct parts, each wrapped in its own <script> tag:
 - The main node definition that is registered with the editor. This defines things such as the palette category, the editable properties

(defaults) and what icon to use. It is within a regular javascript script tag.

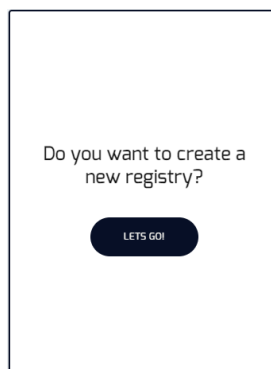
- The edit template that defines the content of the edit dialog for the node. It is defined in a script of type text/html with data-template-name set to the type of the node.
- The help text that gets displayed in the Info sidebar tab. It is defined in a script of type text/html with data-help-name set to the type of the node.

A package.json file is used to package it all together as an npm module. Here is a typical directory structure for a node package:

```
├─ LICENSE
├─ README.md
├─ package.json
├─ examples
│   └─ example-1.json
│       └─ example-2.json
├─ sample
│   └─ icons
│       └─ my-icon.svg
├─ sample.html
└─ sample.js
```

Uploading Nodes in aXet.flows

Once you have created and tested your node, it's time to upload it to **aXet.flows** so that other users of the tool can use it. Access the '*Nodes Libraries*' section and click on the button '*Lets go*'.



Complete the mandatory fields in the pop-up window:

New private library

Main info Extra info NPM Package Upload Community info

Please upload the NPM Package for validate and extract metadata.

The NPM package must meet the conditions of being an NPM project made from the archetype/seed provided by the MD|ONE support team. You can find this archetype through the following Git repository: <https://umane.emeal.nttdata.com/git/ONEPROY/deptapps-flows-contrib-nodes-seed>

+ Choose

DELETE SAVE CANCEL

1. Upload the .json file in which you have packaged the node, and you will see that the fields are automatically filled with your node's information.
2. You must add the following information:
 - a. MD|ONE Project. Users belonging to this project will be able to use and also evolve the shared workflow.
 - b. You can also share your node with the entire .flows community (Optional).
3. In the 'Extra Info' tab, you can modify the library's description.
4. To publish the node, use the 'NPM Package Upload' tab and upload the artifact again.

Subflows

A subflow is a collection of nodes that are collapsed into a single node in the workspace.

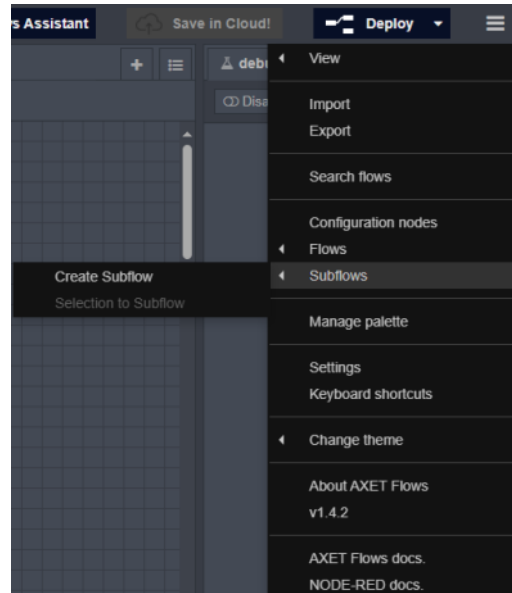
They can be used to reduce some visual complexity of a flow, or to package up a group of nodes as a reusable flow used in multiple places.

Once created, the subflow is added to the palette of available nodes. Individual instances of the subflow can then be added to the workspace just like any other node.

A subflow cannot contain an instance of itself - either directly or indirectly.

Creating an empty subflow

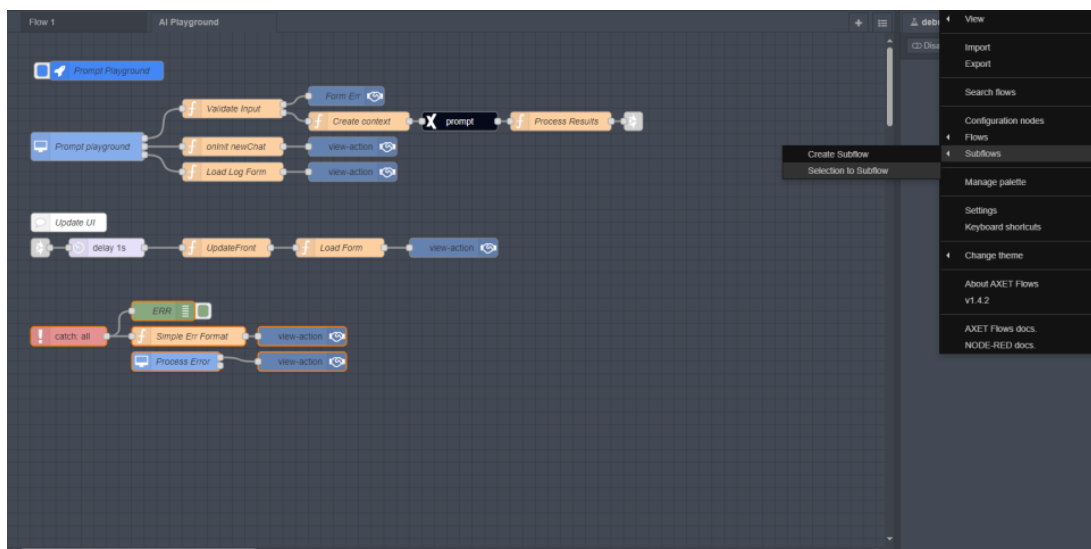
A subflow can be created by selecting the '*Subflow -> Create subflow*' option in the menu. This will create a blank subflow and open it in the workspace.



You can add the components to the workspace to design the subflow.

Converting nodes to a subflow

It is also possible to convert the current selection of nodes to a subflow by selecting the '*Subflow -> Selection to Subflow*' option in the menu. The nodes will be moved to a new subflow and replaced by a subflow instance node within the flow.



Editing a subflow

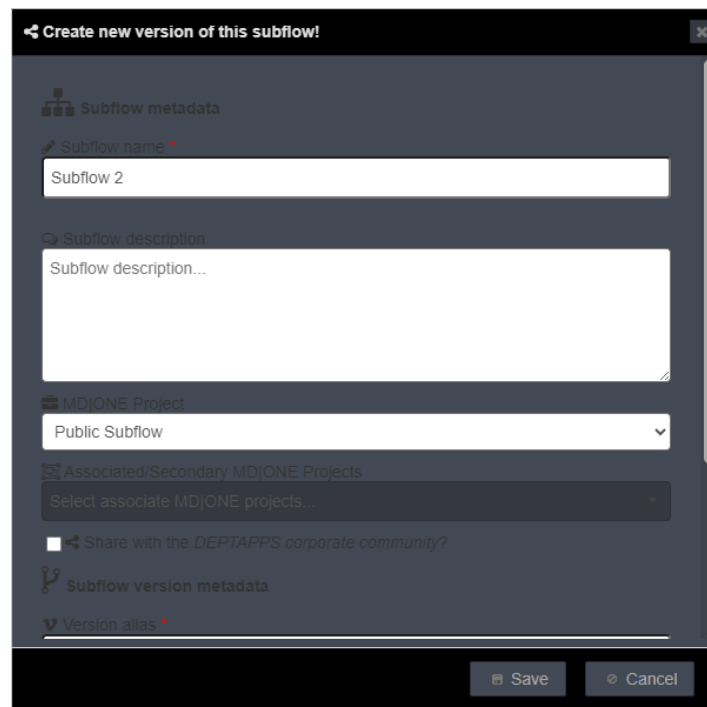
There are two ways to open a subflow to edit its contents. Either double click its node in the palette, or click the *'Edit flow template'* button in the edit dialog of a subflow instance node.

The subflow is opened in the workspace as a new tab. Unlike regular flow tabs, subflow tabs can be closed to hide them.

Share a subflow

Once you have created your subflow, and it has been tested, it is time to share it. The *'Share flow in cloud'* button is available.

By clicking on this button, the process to share the subflow in the **cloud** and make it importable by other development projects is initiated. When the button is pressed, a popup appears where you can enter the necessary data to share the subflow and store it in the cloud.



The screenshot shows a dark-themed dialog box titled "Create new version of this subflow!". It contains several sections for metadata:

- Subflow metadata**: Includes a "Subflow name" field with the text "Subflow 2" and a "Subflow description" field with the placeholder "Subflow description...".
- MDIONE Project**: A dropdown menu currently showing "Public Subflow".
- Associated/Secondary MDIONE Projects**: A button labeled "Select associate MDIONE projects...".
- Share with the DEPTAPPS corporate community?**: A checkbox that is currently unchecked.
- Subflow version metadata**: Includes a "Version alias" field.

At the bottom right, there are "Save" and "Cancel" buttons.

The fields to complete are as follows:

- **Subflow name.** Corresponds to the name that the subflow will have.

Automatically, the name that the subflow has in the designer will be used to share it in the cloud.

- **Subflow description.** More detailed description of the flow, which will also be automatically completed with the information added in the flow designer.
- **MD|ONE project** (mandatory). The shared subflow must be assigned to a project among the projects where the user is registered in the system. Users belonging to this project will be able to use and also evolve the shared flow.
- **Associated MD|ONE projects** (optional). Other projects different from the main project associated with the shared subflow that will allow users associated with these secondary projects to import the subflow in the flow designer. Users belonging to these projects will have access to the shared subflow in "read-only" mode, that is, they will not be able to consolidate new versions of the subflow through the flow designer.
- **Share with the entire DEPTAPPS corporate community?** (optional) If you check this checkbox, any user from **aXet.flows** community will be able to import the subflow in the flow designer. Users belonging to these projects will have access to the shared subflow in "read-only" mode, that is, they will not be able to consolidate new versions of the subflow through the flow designer.
- **Version alias.** Semantic versioning is recommended (<https://semver.org/>).
- **Version description.** Once the node is created/shared in the cloud, you will have access to it, from the palette management (Manage Palette) to be able to install it, although in your case it is already installed.

Once the subflow has been shared, you must take into account two very important aspects:

- If you press the '*Share flow in cloud*' button again on the previously created subflow, a new version will be created.
- You can see that in the designer of the (sub)flow a new button appears, which allows us to execute the '*Unlink from cloud!*' action. This action allows you to unlink the subflow from the cloud, so that if you press the '*Shared flow in cloud!*' button again, a new subflow will be created along with a first version.

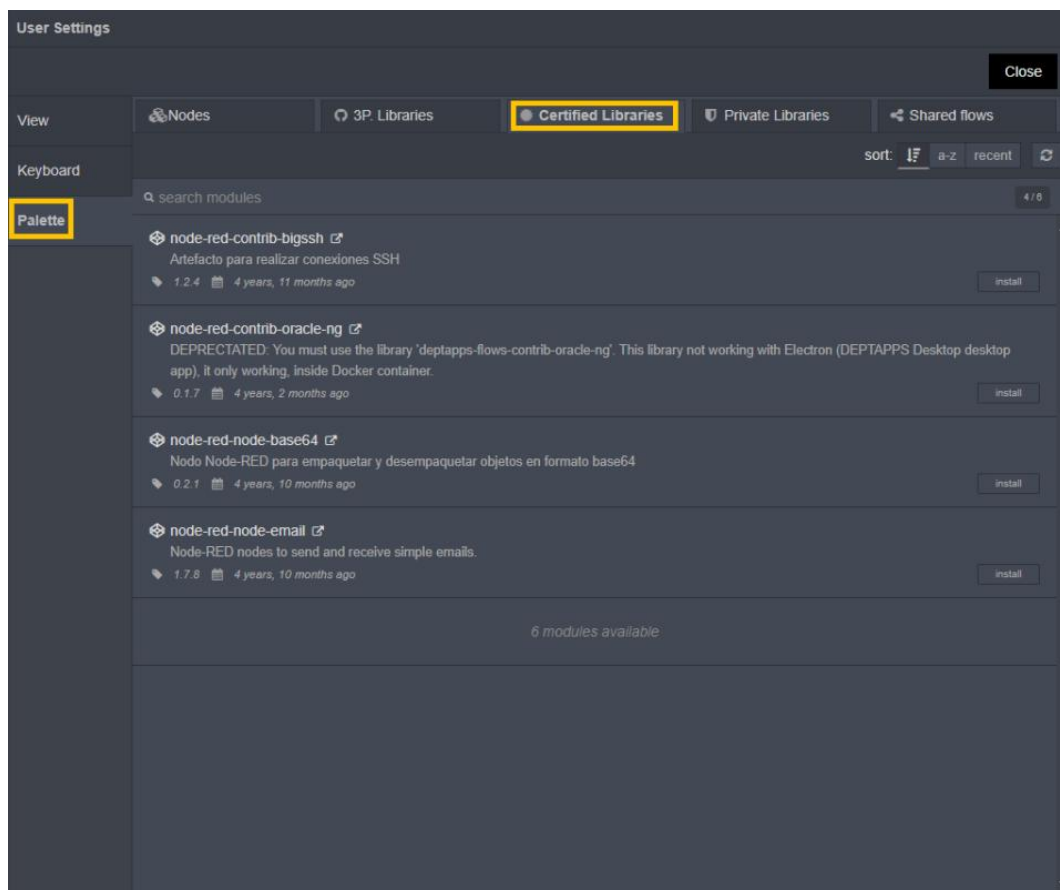
Certified Libraries

aXet.flows is based on NODE-RED, and therefore, you can install any third-party library from the NODE-RED developer community, making it possible to use all the

libraries published at <https://flows.nodered.org/>, and have new nodes available to program your flows. However, these libraries, being open source and developed by the community, may contain undesirable quality, or you may find that the implemented nodes do not work correctly.

The objective of the ‘*Certified Libraries*’ is that the **aXet.flows** team will add information about those libraries/nodes that are considered certified. This means they have previously been installed and tested.

You can use the other third-party libraries, but like any open-source software from the community (see the thousands of repositories that exist on GitHub), it may happen that the node does not function properly or may not suit your needs after using it. You can find them in the **aXet.flows** designer by selecting the ‘*Settings/Palette/Certified Libraries*’ menu.



FAQs

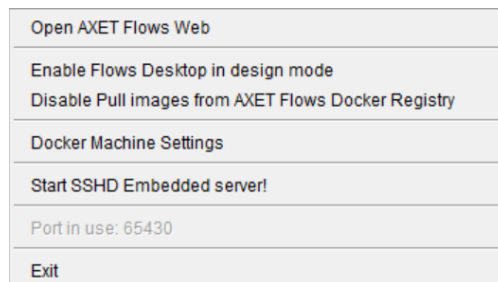
aXet.flows has two versions of the designer. When should I use one or another?

If you are going to develop an application that is a web application, you should use the web designer.

If you are going to develop an application that is a desktop application, you should use the desktop version of the designer.

To select the version you are going to use, open **aXet.flows** context menu in the systray.

Select the menu item *'Enable Flows Desktop in design mode'* if the asset you are going to develop is a desktop application.



By default, the web designer will launch if you do not activate 'Enable Flows Desktop in design mode.'

How to import and export a version in aXet.flows

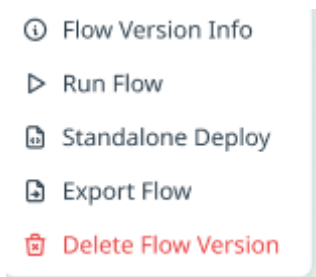
To import a version in **aXet.flows**, you must first have an exported version of a Flow.

To export a version, you must be in a development project that has a saved version.

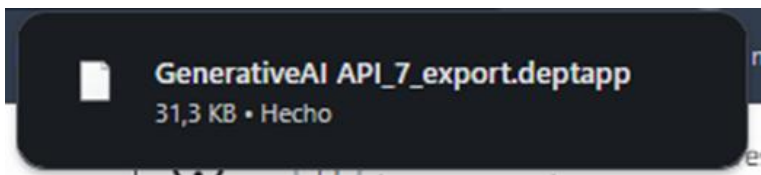
A screenshot of the aXet.flows web interface. On the left is a sidebar with the 'flows' logo and navigation links: Home, Catalog, Nodes libraries, Shared subflows, Digital workers, Blog, Administrator, Users, and Projects. The main area is titled 'Informes Deptapps y TOK' and has buttons for '+ New axet flows' and 'Import version'. Below the title is a 'Filter results' section with a search bar and a 'Discharged Versions' checkbox. A table displays a list of flow versions with columns: Type, Alias, Description, Created date, Created user, Version, and Edit. The table contains five rows of data, all with 'flo' as the alias and 'Terry Vaccaro' as the user. The first row is highlighted with a green checkmark in the Edit column.

Type	Alias	Description	Created date	Created user	Version	Edit
flo	0.7.7	Empty description	28/06/2024 22:18:41	Terry Vaccaro	7.1.3	
flo	0.7.6	Empty description	28/06/2024 07:00:03	Terry Vaccaro	7.1.3	
flo	0.7.5	Empty description	28/06/2024 07:51:46	Terry Vaccaro	7.1.3	
flo	0.7.4	Empty description	28/06/2024 02:44:52	Terry Vaccaro	7.1.3	
	0.7.3	Empty description	28/06/2024 20:08:44	Terry Vaccaro	7.1.3	

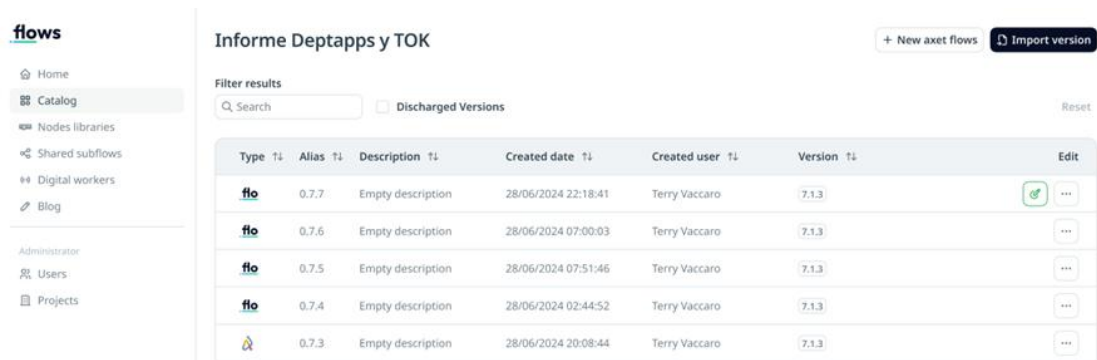
Expand the options of the version and click the *‘Export Flow’* button, as indicated in the following image:



The file with the .deptapp extension of the desired version will be downloaded:



Once the version is downloaded, you must go to your development project where you want to import the version and click the button called *‘Import version’*, as shown in the following image:



After selecting the file to import, a window will open where we can modify the name and description of the version:

Import version 7 of GenerativeAI API Axet Flow

▼ ⓘ Import relevant data

Axet Flow info

- **Name:** GenerativeAI API
- **Creation Date:** 25/01/2024 07:49:48
- **Owner User:** Francisco Javier Romero Yesares (fromeroy@emeal.nttdata.com)
- **Search Tags:** (empty)

Axet Flow version info

- **Alias:** GenerativeAI API
- **Creation Date:** 26/09/2024 14:18:23
- **Creation User:** Francisco Javier Romero Yesares (fromeroy@emeal.nttdata.com)
- [More info...](#)

 Version alias *

☐ Description message

(imported data)

IMPORT

CANCEL

It will ask you for confirmation to lock the development project and it will be created successfully:

Home / Catalog / Prueba Doc

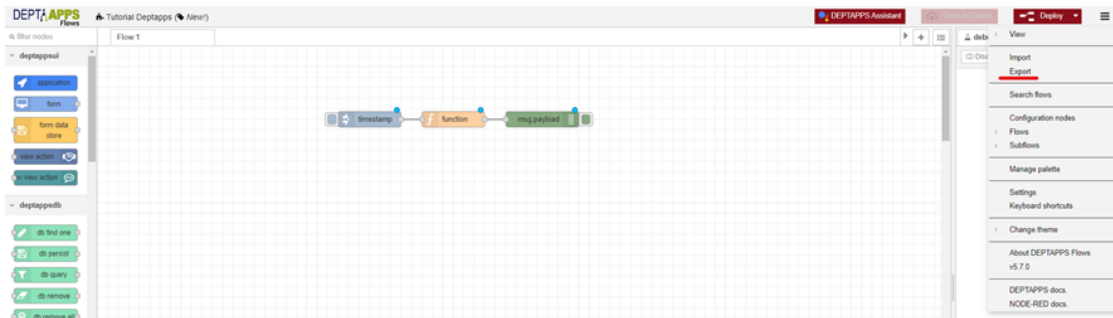
Axet Flow has been unblock successfully!

The Axet Flow version has been imported successfully!

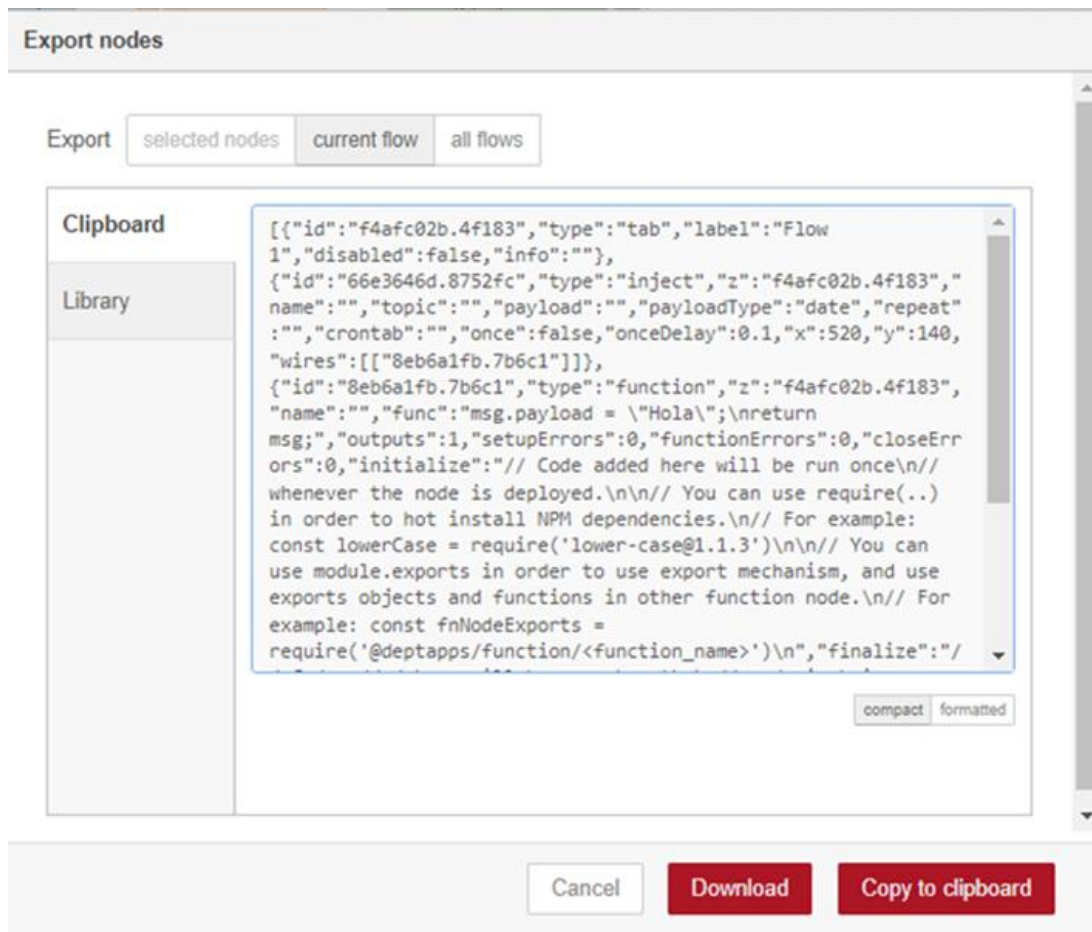
How to import and export a flow in aXet.flows

To import a flow, you must first have an exported flow. Let's see the steps to export a flow.

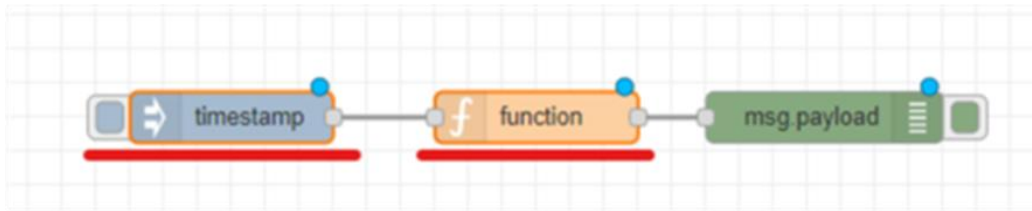
In the Flows designer, within the desired flow tab, go to the options of the flow located in the upper right corner and click on the menu icon, then click on the option called 'Export':



Next, a window will appear indicating the ways in which we can export the flow, either as a file (by clicking the 'Download' button) or by copying the JSON of the flow to the clipboard (by clicking the 'Copy to clipboard' button):

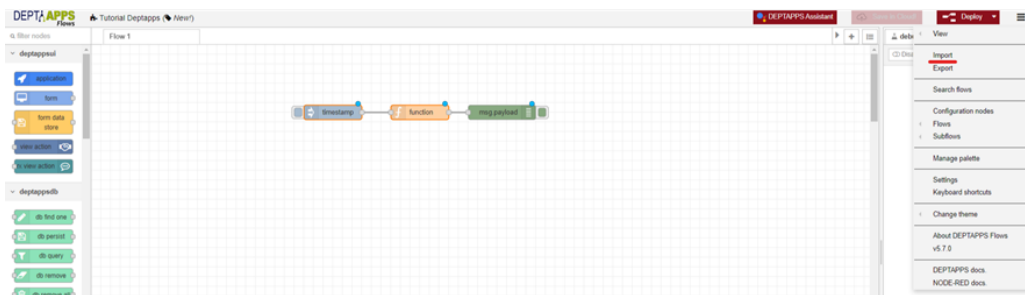


A detail to keep in mind here is that if you do not want to export the entire flow and only want to export specific nodes, this can be done by selecting the nodes either by dragging the cursor over them or by holding down the Ctrl key and clicking on the nodes to be exported. The designer will highlight the selected nodes with an orange border:



These nodes can also be exported by following the same steps as before in the flow export process, and only the selected nodes will be exported.

Once you have chosen one of the export options, the next step is how to import it. To do this, you need to use the designer menu mentioned earlier and select the option called *'Import'*:



A window will open where we can enter the exported flow, either by copying it from the clipboard or by selecting the exported .json file.

The screenshot shows a dialog box titled "Import nodes". It features a sidebar on the left with "Clipboard", "Library", and "Examples" sections. The main area has a text input field for pasting JSON or a file selection button. At the bottom, there are options to import to the "current flow" or "new flow", and "Cancel" and "Import" buttons.

In this window, at the bottom, there is an option called '*Import to*' (shown in the previous image), which indicates whether the imported flow will be placed in the current flow or in a new flow. If current flow is selected, it will copy it to the current flow, and if new flow is selected, it will create a new tab with the imported flow.

Can I pass user credentials through msg?

For security reasons, it is not possible to make this change.

If the development project were allowed to modify the credentials, they would be exposed since flow persistence is done without encryption.

Node creation and how to add created nodes to aXet.flows

To create a node bundle and add a new library or bundle in **aXet.flows**, you must follow the steps indicated in the documentation in Confluence. Here are the steps to follow:

1- [Node bundle creation](#)

2- [Creation of a new library/bundle](#)

Using the view-action and rx-view-action nodes

Unlike the **view-action** node, the **rx-view-action** node sends information to update the view in an unattended manner, without the user having to activate an action by clicking a button. The view will be updated without the user relinquishing control to the server (using WebSocket technology).

Attached is a simple flow in JSON format to import into **aXet.flows**, with an example that shows how the **rx-view-action** node's function performs an action asynchronously to the **view-action** node:

[Example-flow-view-action-and-rx-view-action](#)

Issues with the VPN node?

The 'vpn-connect' / 'vpn-disconnect' nodes are based on the use of the **openconnect** utility, so it is necessary to have this command line interface (CLI) installed:

-Windows: [rg/web/20210505224906/https://support.onevpn.com/windows-10-openconnect](https://support.onevpn.com/windows-10-openconnect). You can download the installer from <https://github.com/openconnect/openconnect-gui/releases>.

-Linux: You can run '*sudo apt install openconnect*' in a command terminal.

Also, keep in mind that:

-VPNs with dual factor authentication do not work with openconnect.

-For faster support, ask the IT team how to connect to the VPN using the openconnect command.

-The node has the ability to import .xml files that contain the configurations. For example, in Cisco, they are located in the following path:

C:\ProgramData\Cisco\Cisco AnyConnect Secure Mobility Client\Profile***.xml

When starting my designer, it shows a message indicating missing nodes to install

Don't worry, it is not an error, you just need to wait a little longer because those nodes are being installed.

It is recommended that after a few seconds or a minute, refresh the designer to verify that the nodes have been installed correctly and that they are in their proper places.

aXet.flows load in Docker mode but not in Electron mode, what can I do?

If this happens, you should delete the **.node-red** directory located in the user folder of your computer.

Once the directory is deleted, you need to run **aXet.flows** in Electron mode again. After that, the flows should be displayed correctly.

Additionally, you should check the version of NodeJS and NPM installed on your computer, as an incompatible version of either of them could be causing the error. The recommended version of NodeJS is 12.18.4, with a compatible version of NPM being 6.14.6.

The exportable flows generated via Electron are not opening, what can I do?

Probably the flows are not being loaded in **aXet.flows**. Before generating the exportable, you should check if the flows are being loaded correctly. If they are not, you should delete the **.node-red** directory located in the user folder of your computer.

Once the directory is deleted, you need to run the **aXet.flows** again in Electron mode. After that, the flows should be displayed correctly and the exportable for **aXet.flows** can be generated.

Additionally, you should check the version of NodeJS and NPM installed on your computer, as an incompatible version of either of them could be causing the error. The minimum recommended version of NodeJS is 12.18.4, with a compatible version of NPM being 6.14.6.

Glossary

This is a glossary of relevant terms:

- **Node**

A Node is the basic building block of a flow.

Nodes are triggered by either receiving a message from the previous node in a flow, or by waiting for some external event, such as an incoming HTTP request, a timer or GPIO hardware change. They process that message, or event, and then may send a message to the next nodes in the flow.

A node can have at most one input port and as many output ports as it requires.

- **Configuration node**

A Configuration (config) Node is a special type of node that holds reusable configuration that can be shared by regular nodes in a flow.

For example, the MQTT In and Out nodes use an MQTT Broker config node to represent a shared connection to an MQTT broker.

Config nodes do not appear in the main workspace, but can be seen by opening the Configuration nodes sidebar.

- **Flow**

A Flow is represented as a tab within the editor workspace and is the main way to organise nodes.

The term “flow” is also used to informally describe a single set of connected nodes. So a flow (tab) can contain multiple flows (sets of connected nodes).

- **Context**

Context is a way to store information that can be shared between nodes without using the messages that pass through a flow.

There are three types of context:

Node - only visible to the node that set the value

Flow - visible to all nodes on the same flow (or tab in the editor)

Global - visible to all nodes

By default, **aXet.flows** uses an in-memory Context store so values do not get saved across restarts. It can be configured to use a file-system based

store to make the values persistent. It is also possible to plug-in alternative storage plugins.

- **Message**

Messages are what pass between the nodes in a flow. They are plain JavaScript objects that can have any set of properties. They are often referred to as msg within the editor.

By convention, they have a “payload” property containing the most useful information.

- **Subflow**

A Subflow is a collection of nodes that are collapsed into a single node in the workspace.

They can be used to reduce some visual complexity of a flow, or to package up a group of nodes as a reusable component used in multiple places.

- **Wire**

Wires connect the nodes and represent how messages pass through the flow.

- **Palette**

The Palette is on the left of the editor and lists of the nodes that are available to use in flows.

Extra nodes can be installed into the palette using either the command-line or the Palette Manager.

- **Workspace**

The Workspace is the main area where flows are developed by dragging nodes from the palette and wiring them together.

The workspace has a row of tabs along the top; one for each flow and any subflows that have been opened.

- **Sidebar**

The sidebar contains panels that provide a number of useful tools within the editor. These include panels to view more information and help about a node, to view debug message and to view the flow’s configuration nodes.